

healthcare (1)

June 24, 2026

1 Healthcare Data Analytics Project

1.1 Project Overview

This project analyzes real-world healthcare datasets from CMS (Centers for Medicare & Medicaid Services) to identify trends in:

- Hospital complications and deaths
- Patient satisfaction
- Medicare spending
- Timely and effective care

The project uses Python, pandas, NumPy, and matplotlib for data cleaning, exploratory data analysis (EDA), visualization, correlation analysis, and regression analysis.

1.2 Business Questions

- Do hospitals with higher Medicare spending achieve higher patient satisfaction?
- Which states show the highest complication and outcome scores?
- How does patient satisfaction vary across hospitals and states?
- Are there measurable relationships between healthcare spending and healthcare quality?

1.3 Technologies Used

- Python
- Pandas
- NumPy
- Matplotlib
- Google Colab
- Data Cleaning
- Exploratory Data Analysis (EDA)
- Correlation Analysis
- Regression Analysis
- Data Visualization

2 Data Collection & Dataset Loading

In this section, we load multiple CMS healthcare datasets containing: - Hospital complications and mortality - Patient satisfaction surveys - Medicare spending data - Timely and effective care

metrics

These datasets will be cleaned and analyzed throughout the project.

```
[ ]: # Core libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Display settings
pd.set_option('display.max_columns', None)

# Load datasets
complications_df = pd.read_csv('/content/complications_and_deaths_hospital.csv')

hcahps_df = pd.read_csv(
    '/content/hcahps_hospital.csv',
    low_memory=False
)

medicare_df = pd.read_csv(
    '/content/medicare_hospital_spending_per_patient_hospital.csv'
)

timely_df = pd.read_csv(
    '/content/timely_and_effective_care_hospital.csv'
)

# Verify datasets loaded
print("Complications shape:", complications_df.shape)
print("HCAHPS shape:", hcahps_df.shape)
print("Medicare shape:", medicare_df.shape)
print("Timely & Effective Care shape:", timely_df.shape)
```

Complications shape: (90801, 18)

HCAHPS shape: (444447, 22)

Medicare shape: (4615, 14)

Timely & Effective Care shape: (118520, 16)

```
[ ]: # Preview first rows
complications_df.head()
```

```
[ ]:  facility_id      facility_name      address \
0      10001.0  SOUTHEAST HEALTH MEDICAL CENTER  1108 ROSS CLARK CIRCLE
1      10001.0  SOUTHEAST HEALTH MEDICAL CENTER  1108 ROSS CLARK CIRCLE
2      10001.0  SOUTHEAST HEALTH MEDICAL CENTER  1108 ROSS CLARK CIRCLE
3      10001.0  SOUTHEAST HEALTH MEDICAL CENTER  1108 ROSS CLARK CIRCLE
4      10001.0  SOUTHEAST HEALTH MEDICAL CENTER  1108 ROSS CLARK CIRCLE
```

	citytown	state	zip_code	countyparish	telephone_number	measure_id	\
0	DOTHAN	AL	36301	HOUSTON	(334) 793-8701	COMP_HIP_KNEE	
1	DOTHAN	AL	36301	HOUSTON	(334) 793-8701	MORT_30_AMI	
2	DOTHAN	AL	36301	HOUSTON	(334) 793-8701	MORT_30_CABG	
3	DOTHAN	AL	36301	HOUSTON	(334) 793-8701	MORT_30_COPD	
4	DOTHAN	AL	36301	HOUSTON	(334) 793-8701	MORT_30_HF	

	measure_name	\
0	Rate of complications for hip/knee replacement...	
1	Death rate for heart attack patients	
2	Death rate for CABG surgery patients	
3	Death rate for COPD patients	
4	Death rate for heart failure patients	

	compared_to_national	denominator	score	lower_estimate	\
0	No Different Than the National Rate	32.0	3.0	1.6	
1	No Different Than the National Rate	291.0	10.8	8.4	
2	No Different Than the National Rate	157.0	3.8	2.1	
3	No Different Than the National Rate	122.0	7.9	5.2	
4	No Different Than the National Rate	610.0	10.5	8.6	

	higher_estimate	footnote	start_date	end_date
0	5.5	NaN	2020-07-01	2023-03-31
1	13.6	NaN	2020-07-01	2023-06-30
2	6.8	NaN	2020-07-01	2023-06-30
3	11.9	NaN	2020-07-01	2023-06-30
4	12.9	NaN	2020-07-01	2023-06-30

```
[ ]: # Dataset info
      complications_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 90801 entries, 0 to 90800
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   facility_id            87685 non-null  float64
1   facility_name          90801 non-null  object
2   address                90801 non-null  object
3   citytown               90801 non-null  object
4   state                  90801 non-null  object
5   zip_code               90801 non-null  int64
6   countyparish           90801 non-null  object
7   telephone_number      90801 non-null  object
8   measure_id             90801 non-null  object
9   measure_name           90801 non-null  object
```

```

10 compared_to_national  90801 non-null  object
11 denominator          45970 non-null  float64
12 score                 48934 non-null  float64
13 lower_estimate        48934 non-null  float64
14 higher_estimate       48934 non-null  float64
15 footnote              42306 non-null  float64
16 start_date            90801 non-null  object
17 end_date              90801 non-null  object
dtypes: float64(6), int64(1), object(11)
memory usage: 12.5+ MB

```

3 Data Cleaning & Preparation

Healthcare datasets often contain: - Missing values - Incorrect datatypes - Duplicate records - Inconsistent formatting

This section prepares the datasets for accurate analysis.

```
[ ]: # Check missing values
complications_df.isnull().sum()
```

```
[ ]: facility_id          3116
facility_name            0
address                 0
citytown                0
state                   0
zip_code                0
countyparish            0
telephone_number        0
measure_id              0
measure_name            0
compared_to_national    0
denominator             44831
score                   41867
lower_estimate          41867
higher_estimate         41867
footnote                48495
start_date              0
end_date                0
dtype: int64

```

```
[ ]: # Numeric columns to convert
numeric_cols = [
    'denominator',
    'score',
    'lower_estimate',
    'higher_estimate'
]
```

```

# Convert safely
for col in numeric_cols:
    complications_df[col] = pd.to_numeric(
        complications_df[col],
        errors='coerce'
    )

# Verify datatypes
complications_df[numeric_cols].dtypes

```

```

[ ]: denominator    float64
     score          float64
     lower_estimate float64
     higher_estimate float64
     dtype: object

```

```

[ ]: # Convert date columns
complications_df['start_date'] = pd.to_datetime(
    complications_df['start_date'],
    errors='coerce'
)

complications_df['end_date'] = pd.to_datetime(
    complications_df['end_date'],
    errors='coerce'
)

# Verify datatypes
complications_df[['start_date', 'end_date']].dtypes

```

```

[ ]: start_date    datetime64[ns]
     end_date      datetime64[ns]
     dtype: object

```

```

[ ]: # Check duplicate rows
duplicate_count = complications_df.duplicated().sum()

print("Duplicate rows:", duplicate_count)

```

Duplicate rows: 0

```

[ ]: # Fill missing numeric values with 0
numeric_cols = [
    'denominator',
    'score',
    'lower_estimate',

```

```

        'higher_estimate'
    ]

    complications_df[numeric_cols] = complications_df[
        numeric_cols
    ].fillna(0)

    # Fill missing text/object values with 'Unknown'
    text_cols = complications_df.select_dtypes(
        include='object'
    ).columns

    complications_df[text_cols] = complications_df[
        text_cols
    ].fillna('Unknown')

    # Verify missing values
    complications_df.isnull().sum()

```

```

[ ]: facility_id          3116
    facility_name         0
    address               0
    citytown              0
    state                 0
    zip_code              0
    countyparish          0
    telephone_number      0
    measure_id            0
    measure_name          0
    compared_to_national  0
    denominator           0
    score                 0
    lower_estimate        0
    higher_estimate       0
    footnote              48495
    start_date            0
    end_date              0
    dtype: int64

```

```

[ ]: # Strip whitespace from text columns
    text_cols = complications_df.select_dtypes(
        include='object'
    ).columns

    for col in text_cols:
        complications_df[col] = complications_df[col].str.strip()

```

```
# Create cleaned dataframe copy
complications_cleaned_df = complications_df.copy()

# Verify dataset info
complications_cleaned_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 90801 entries, 0 to 90800
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   facility_id                           87685 non-null  float64
1   facility_name                         90801 non-null  object
2   address                              90801 non-null  object
3   citytown                             90801 non-null  object
4   state                                90801 non-null  object
5   zip_code                             90801 non-null  int64
6   countyparish                         90801 non-null  object
7   telephone_number                    90801 non-null  object
8   measure_id                           90801 non-null  object
9   measure_name                         90801 non-null  object
10  compared_to_national                 90801 non-null  object
11  denominator                          90801 non-null  float64
12  score                                90801 non-null  float64
13  lower_estimate                       90801 non-null  float64
14  higher_estimate                      90801 non-null  float64
15  footnote                             42306 non-null  float64
16  start_date                           90801 non-null  datetime64[ns]
17  end_date                             90801 non-null  datetime64[ns]
dtypes: datetime64[ns](2), float64(6), int64(1), object(9)
memory usage: 12.5+ MB
```

```
[ ]: # Save cleaned dataset
complications_cleaned_df.to_csv(
    '/content/complications_cleaned_final.csv',
    index=False
)

print("Cleaned complications dataset saved.")
```

Cleaned complications dataset saved.

```
[ ]: # Summary statistics for numeric columns
complications_cleaned_df[
    ['score', 'denominator', 'lower_estimate', 'higher_estimate']
].describe()
```

```
[ ]:      score    denominator  lower_estimate  higher_estimate
count  90801.000000  90801.000000    90801.000000    90801.000000
mean      6.034206    598.313862      3.997938      8.285794
std      23.531840   1902.382486     17.149939     30.103884
min       0.000000     0.000000     0.000000     0.000000
25%       0.000000     0.000000     0.000000     0.000000
50%       0.270000     26.000000     0.000000     0.490000
75%       3.660000    275.000000     1.160000     6.260000
max      258.280000   57005.000000    220.220000    302.620000
```

4 Exploratory Data Analysis (EDA)

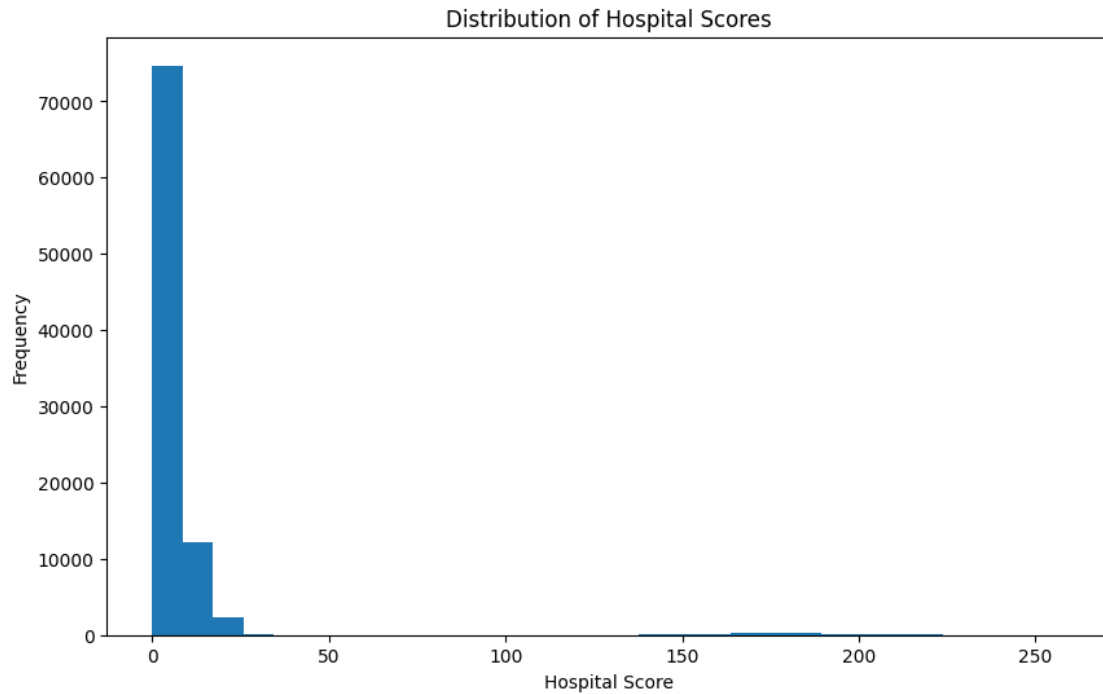
This section explores: - Distribution of hospital scores - Outliers - Correlations between healthcare metrics - State-level healthcare performance

```
[ ]: # Histogram of hospital scores
plt.figure(figsize=(10,6))

plt.hist(
    complications_cleaned_df['score'],
    bins=30
)

# Labels and title
plt.xlabel('Hospital Score')
plt.ylabel('Frequency')
plt.title('Distribution of Hospital Scores')

plt.show()
```

```
[ ]: # Calculate IQR for outlier detection
Q1 = complications_cleaned_df['score'].quantile(0.25)
Q3 = complications_cleaned_df['score'].quantile(0.75)

IQR = Q3 - Q1

# Define outlier bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Identify outliers
outliers = complications_cleaned_df[
    (complications_cleaned_df['score'] < lower_bound) |
    (complications_cleaned_df['score'] > upper_bound)
]

# Display results
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)
print("Number of Outliers:", outliers.shape[0])

outliers[
    ['facility_name', 'citytown', 'state',
     'measure_name', 'score']
```

```
] .head(15)
```

Lower Bound: -5.49

Upper Bound: 9.15

Number of Outliers: 15321

```
[ ]:          facility_name  citytown state \
1  SOUTHEAST HEALTH MEDICAL CENTER  DOTHAN  AL
4  SOUTHEAST HEALTH MEDICAL CENTER  DOTHAN  AL
5  SOUTHEAST HEALTH MEDICAL CENTER  DOTHAN  AL
6  SOUTHEAST HEALTH MEDICAL CENTER  DOTHAN  AL
8  SOUTHEAST HEALTH MEDICAL CENTER  DOTHAN  AL
13 SOUTHEAST HEALTH MEDICAL CENTER  DOTHAN  AL
20      MARSHALL MEDICAL CENTERS    BOAZ  AL
22      MARSHALL MEDICAL CENTERS    BOAZ  AL
23      MARSHALL MEDICAL CENTERS    BOAZ  AL
24      MARSHALL MEDICAL CENTERS    BOAZ  AL
25      MARSHALL MEDICAL CENTERS    BOAZ  AL
27      MARSHALL MEDICAL CENTERS    BOAZ  AL
32      MARSHALL MEDICAL CENTERS    BOAZ  AL
39  NORTH ALABAMA MEDICAL CENTER  FLORENCE  AL
41  NORTH ALABAMA MEDICAL CENTER  FLORENCE  AL

          measure_name  score
1      Death rate for heart attack patients  10.80
4      Death rate for heart failure patients  10.50
5      Death rate for pneumonia patients  18.90
6      Death rate for stroke patients  13.70
8  Death rate among surgical inpatients with seri...  194.78
13      Postoperative respiratory failure rate  12.33
20      Death rate for heart attack patients  14.50
22      Death rate for COPD patients  9.20
23      Death rate for heart failure patients  12.70
24      Death rate for pneumonia patients  23.10
25      Death rate for stroke patients  13.90
27  Death rate among surgical inpatients with seri...  191.14
32      Postoperative respiratory failure rate  10.11
39      Death rate for heart attack patients  16.60
41      Death rate for COPD patients  10.20
```

```
[ ]: # Boxplot of hospital scores
plt.figure(figsize=(8,6))

plt.boxplot(
    complications_cleaned_df['score']
)
```

```
# Labels and title
plt.ylabel('Hospital Score')
plt.title('Boxplot of Hospital Scores')

plt.show()
```



```
[ ]: # Correlation matrix
correlation_matrix = complications_cleaned_df[
    ['score', 'denominator', 'lower_estimate', 'higher_estimate']
].corr()

# Display correlation matrix
correlation_matrix
```

```
[ ]:
```

	score	denominator	lower_estimate	higher_estimate
score	1.000000	-0.049875	0.994773	0.998116
denominator	-0.049875	1.000000	-0.048333	-0.051802
lower_estimate	0.994773	-0.048333	1.000000	0.986853
higher_estimate	0.998116	-0.051802	0.986853	1.000000

```
[ ]: # Heatmap of correlation matrix
plt.figure(figsize=(8,6))

plt.imshow(
    correlation_matrix,
    cmap='coolwarm',
    interpolation='nearest'
)

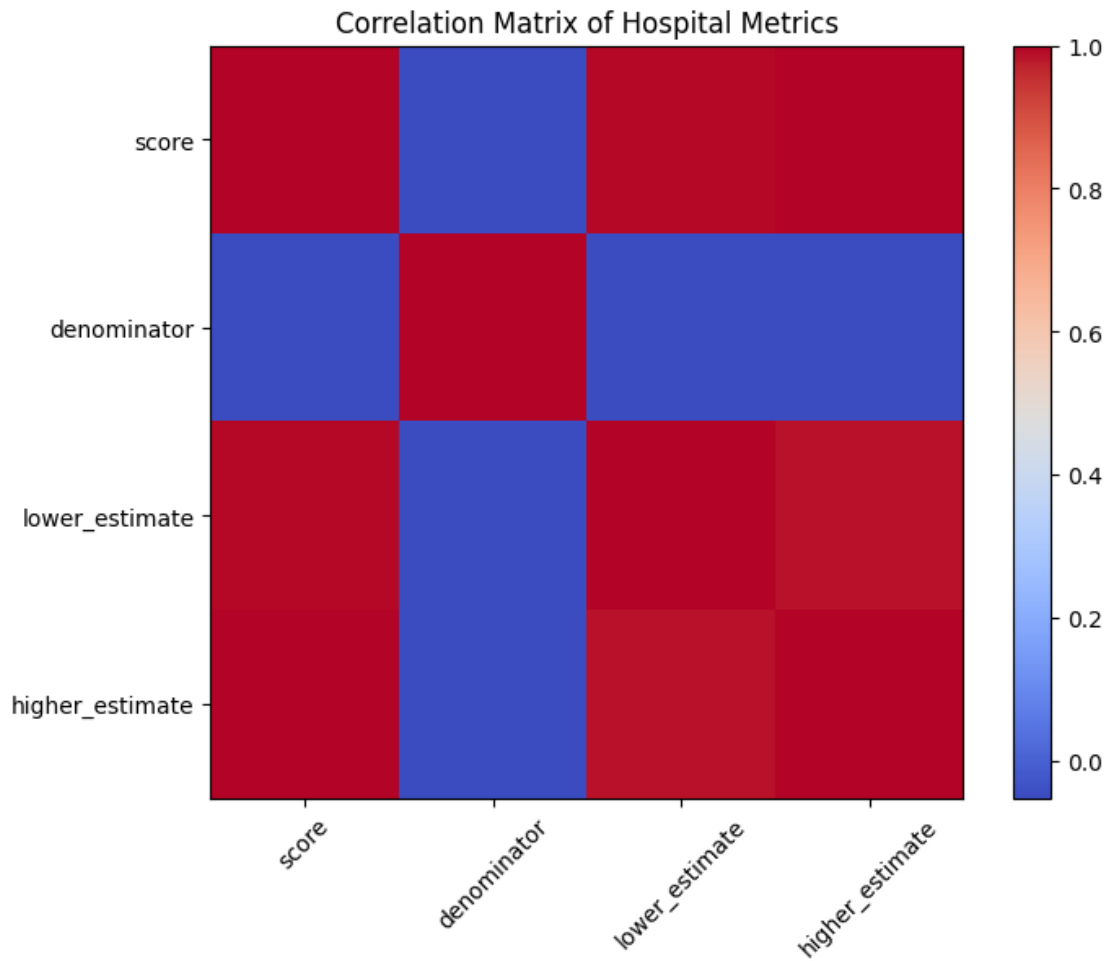
# Labels
plt.xticks(
    range(len(correlation_matrix.columns)),
    correlation_matrix.columns,
    rotation=45
)

plt.yticks(
    range(len(correlation_matrix.columns)),
    correlation_matrix.columns
)

# Color bar
plt.colorbar()

# Title
plt.title('Correlation Matrix of Hospital Metrics')

plt.show()
```



```
[ ]: # Average score by measure
measure_avg_scores = complications_cleaned_df.groupby(
    'measure_name'
)['score'].mean().sort_values(ascending=False)

# Top 10 highest average complication measures
top_measures = measure_avg_scores.head(10)

# Display results
top_measures
```

```
[ ]: measure_name
Death rate among surgical inpatients with serious treatable complications
58.375049
Death rate for pneumonia patients
13.692237
Death rate for heart failure patients
```

```

7.847772
Death rate for stroke patients
6.376313
Postoperative respiratory failure rate
5.523963
Death rate for heart attack patients
5.281440
Death rate for COPD patients
5.232392
Postoperative sepsis rate
2.892348
Perioperative pulmonary embolism or deep vein thrombosis rate
2.350465
Postoperative hemorrhage or hematoma rate
1.467625
Name: score, dtype: float64

```

```

[ ]: # Bottom 10 lowest average complication measures
lowest_measures = measure_avg_scores.tail(10)

# Display results
lowest_measures

```

```

[ ]: measure_name
Postoperative hemorrhage or hematoma rate                1.467625
Rate of complications for hip/knee replacement patients  1.252542
Postoperative wound dehiscence rate                      0.992911
Postoperative acute kidney injury requiring dialysis rate 0.889688
CMS Medicare PSI 90: Patient safety and adverse events composite 0.619734
Death rate for CABG surgery patients                    0.545323
Abdominopelvic accidental puncture or laceration rate    0.534308
Pressure ulcer rate                                     0.417702
In-hospital fall-associated fracture rate                 0.192306
Iatrogenic pneumothorax rate                             0.165790
Name: score, dtype: float64

```

```

[ ]: import matplotlib.pyplot as plt

# Create figure
fig, axes = plt.subplots(1, 2, figsize=(16,6))

# Top measures chart
axes[0].barh(top_measures.index, top_measures.values)
axes[0].set_title('Top 10 Highest Average Hospital Complication Measures')
axes[0].set_xlabel('Average Score')
axes[0].invert_yaxis()

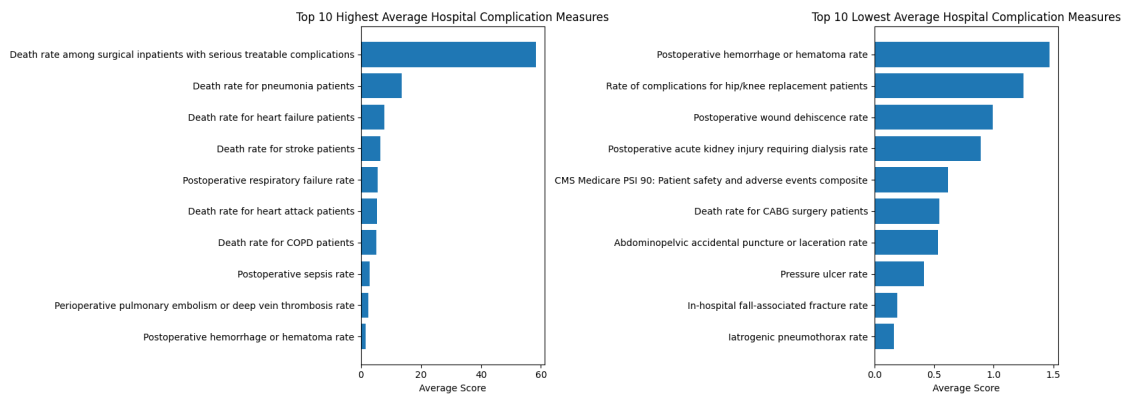
```

```

# Lowest measures chart
axes[1].barh(lowest_measures.index, lowest_measures.values)
axes[1].set_title('Top 10 Lowest Average Hospital Complication Measures')
axes[1].set_xlabel('Average Score')
axes[1].invert_yaxis()

plt.tight_layout()
plt.show()

```



5 State-Level Healthcare Analysis

This section identifies: - Highest-risk states - Lowest-risk states - Regional healthcare trends - State performance comparisons

```

[ ]: # Average score by state
state_avg_scores = complications_cleaned_df.groupby(
    'state'
)['score'].mean().sort_values(ascending=False)

# Top 10 highest-risk states
top_10_states = state_avg_scores.head(10)

# Bottom 10 lowest-risk states
bottom_10_states = state_avg_scores.tail(10)

# Display results
top_10_states, bottom_10_states

```

```

[ ]: (state
      NJ      11.309093
      MD      10.741469
      DC      10.497368
      FL       9.769033

```

```
CT      9.392368
MA      8.825279
NY      8.159693
VA      8.143533
SC      7.827985
DE      7.672105
Name: score, dtype: float64,
state
MN      2.954952
MP      2.863158
WY      2.862468
VI      2.697368
ND      2.674082
MT      2.542029
SD      2.085386
GU      1.936842
PR      0.733512
AS      0.000000
Name: score, dtype: float64)
```

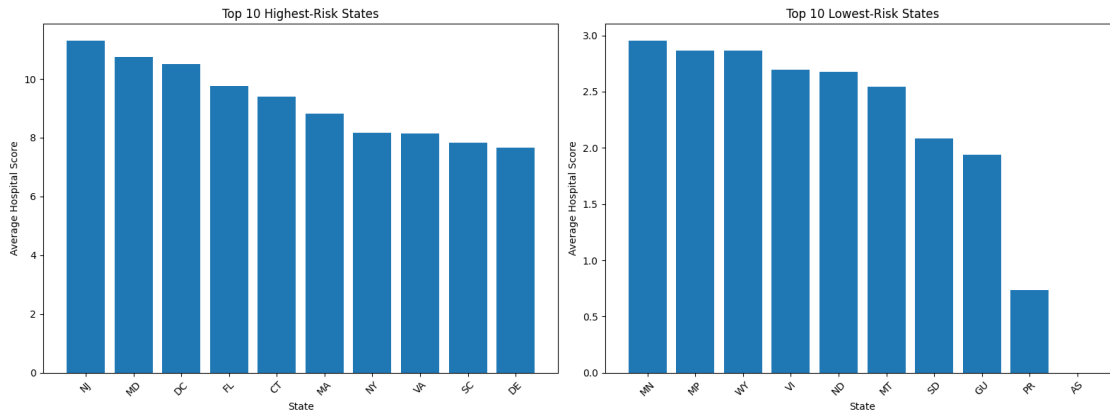
```
[ ]: import matplotlib.pyplot as plt

# Create figure
fig, axes = plt.subplots(1, 2, figsize=(16,6))

# Highest-risk states
axes[0].bar(top_10_states.index, top_10_states.values)
axes[0].set_title('Top 10 Highest-Risk States')
axes[0].set_xlabel('State')
axes[0].set_ylabel('Average Hospital Score')
axes[0].tick_params(axis='x', rotation=45)

# Lowest-risk states
axes[1].bar(bottom_10_states.index, bottom_10_states.values)
axes[1].set_title('Top 10 Lowest-Risk States')
axes[1].set_xlabel('State')
axes[1].set_ylabel('Average Hospital Score')
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```

```
[ ]: # Average hospital scores by facility
hospital_scores = complications_cleaned_df.groupby(
    'facility_name'
)['score'].mean().sort_values(ascending=False)

# Top 10 highest-risk hospitals
top_hospitals_avg = hospital_scores.head(10)

# Bottom 10 lowest-risk hospitals
bottom_hospitals_avg = hospital_scores.tail(10)

# Display results
top_hospitals_avg, bottom_hospitals_avg
```

```
[ ]: (facility_name
      JACKSON HEALTH SYSTEM                19.642632
      MARION COMMUNITIY HOSPITAL           19.489474
      UNITED HEALTH SERVICES HOSPITALS, INC 19.344211
      MERCYONE SIOUXLAND MEDICAL CENTER     18.863684
      HUNTSVILLE HOSPITAL                 18.698947
      NORTHEAST ALABAMA REGIONAL MEDICAL CENTER 18.570000
      LEGACY EMANUEL MEDICAL CENTER         18.545789
      OWENSBORO HEALTH REGIONAL HOSPITAL     18.495263
      NORMAN REGIONAL                     18.423158
      BUTLER MEMORIAL HOSPITAL              18.370000
      Name: score, dtype: float64,
      facility_name
      TEXAS CHILDRENS HOSP                  0.0
      KAHUKU MEDICAL CENTER                 0.0
      JEWELL COUNTY HOSPITAL                0.0
      FRANCISCAN BEACON HOSPITAL, LLC       0.0
      JOHNSON COUNTY COMMUNITY HOSPITAL     0.0)
```

OTHELLO COMMUNITY HOSPITAL	0.0
MARY BRIDGE CHILDREN'S HOSPITAL	0.0
MASON DISTRICT HOSPITAL	0.0
MARLETTE REGIONAL HOSPITAL	0.0
TETON VALLEY HOSPITAL	0.0

Name: score, dtype: float64)

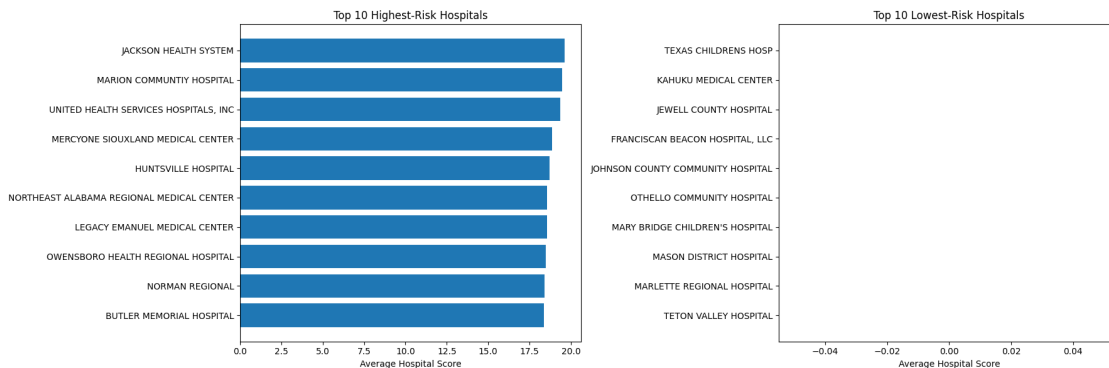
```
[ ]: import matplotlib.pyplot as plt

# Create figure
fig, axes = plt.subplots(1, 2, figsize=(18,6))

# Highest-risk hospitals
axes[0].barh(top_hospitals_avg.index, top_hospitals_avg.values)
axes[0].set_title('Top 10 Highest-Risk Hospitals')
axes[0].set_xlabel('Average Hospital Score')
axes[0].invert_yaxis()

# Lowest-risk hospitals
axes[1].barh(bottom_hospitals_avg.index, bottom_hospitals_avg.values)
axes[1].set_title('Top 10 Lowest-Risk Hospitals')
axes[1].set_xlabel('Average Hospital Score')
axes[1].invert_yaxis()

plt.tight_layout()
plt.show()
```



6 Time Trend Analysis

This section analyzes how hospital complication scores changed over time and identifies trends among high-risk states and healthcare measures.

```
[ ]: # Extract year from start_date
complications_cleaned_df['year'] = complications_cleaned_df['start_date'].dt.
    ↪ year

# Average score by year
yearly_scores = complications_cleaned_df.groupby(
    'year'
)['score'].mean()

# Display results
yearly_scores
```

```
[ ]: year
2020    5.746860
2021    6.201824
Name: score, dtype: float64
```

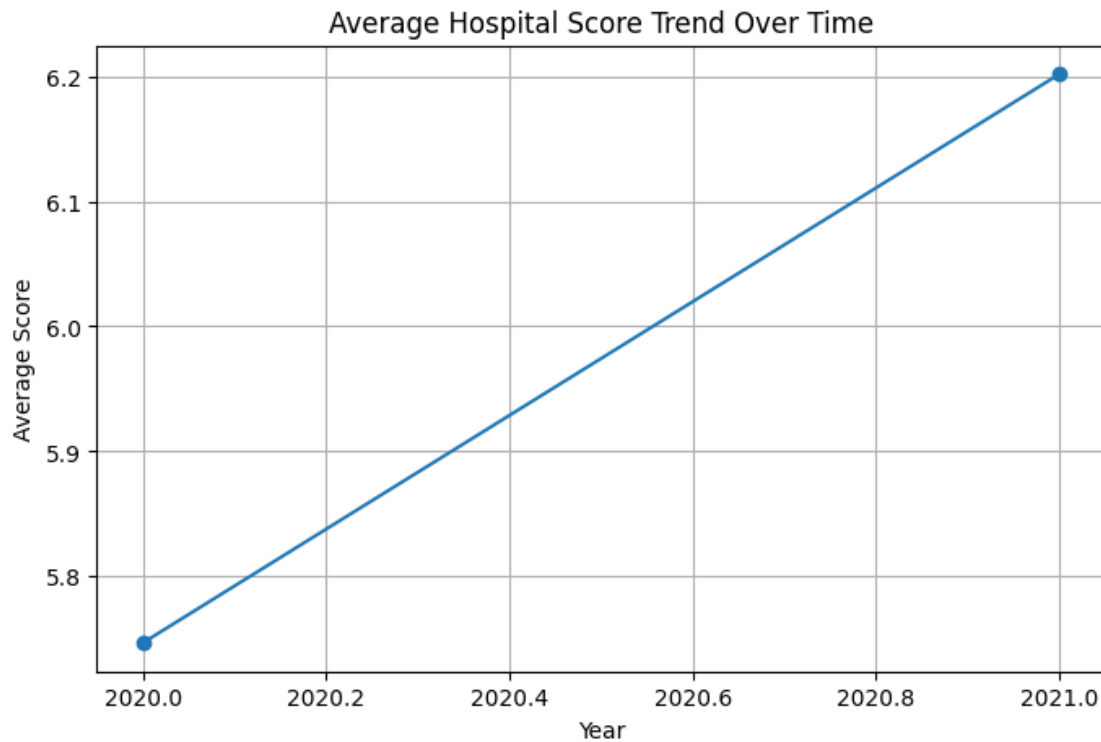
```
[ ]: # Plot yearly hospital score trends
plt.figure(figsize=(8,5))

plt.plot(
    yearly_scores.index,
    yearly_scores.values,
    marker='o'
)

plt.title('Average Hospital Score Trend Over Time')
plt.xlabel('Year')
plt.ylabel('Average Score')

plt.grid(True)

plt.show()
```



```
[ ]: # High-risk states to analyze
high_risk_states = ['NJ', 'MD', 'DC', 'FL', 'CT']

# Filter dataframe
high_risk_df = complications_cleaned_df[
    complications_cleaned_df['state'].isin(high_risk_states)
]

# Average score by year and state
state_trends = high_risk_df.groupby(
    ['year', 'state']
)['score'].mean().reset_index()

# Display results
state_trends.head()
```

```
[ ]:   year state    score
0  2020    CT  7.626667
1  2020    DC  6.850000
2  2020    FL  7.906881
3  2020    MD  8.093155
4  2020    NJ  8.583077
```

```
[ ]: # Plot trends for high-risk states
plt.figure(figsize=(10,6))

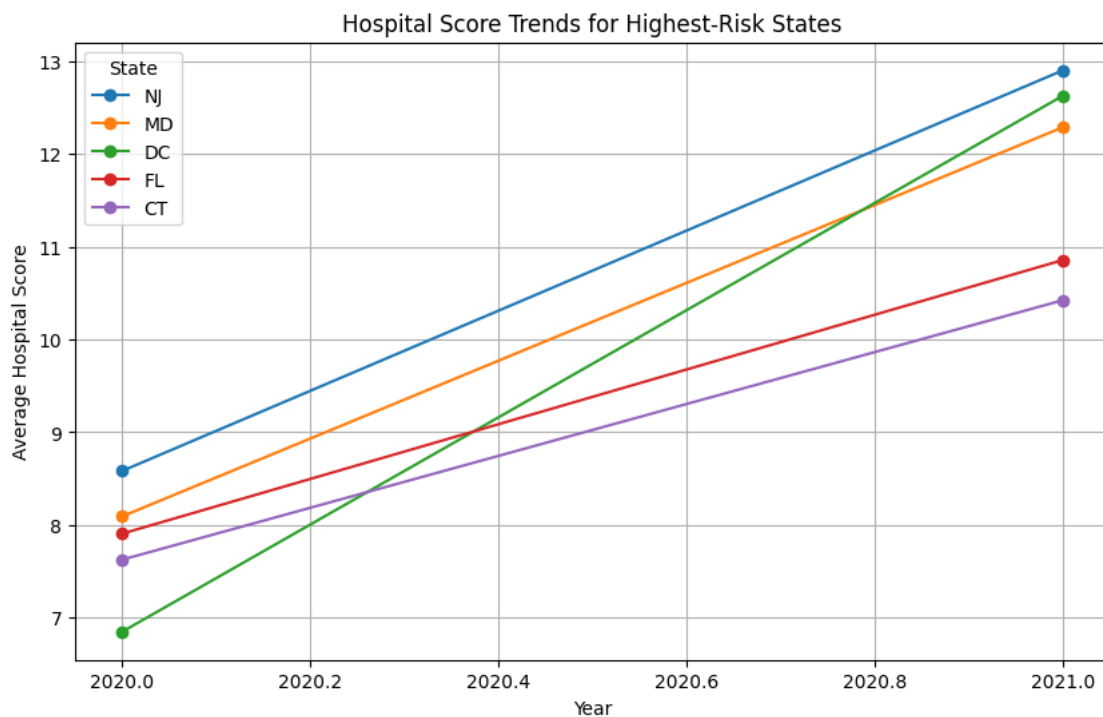
for state in high_risk_states:
    state_data = state_trends[state_trends['state'] == state]

    plt.plot(
        state_data['year'],
        state_data['score'],
        marker='o',
        label=state
    )

plt.title('Hospital Score Trends for Highest-Risk States')
plt.xlabel('Year')
plt.ylabel('Average Hospital Score')

plt.legend(title='State')
plt.grid(True)

plt.show()
```



```
[ ]: # Average score by year and complication measure
measure_trends = complications_cleaned_df.groupby(
    ['year', 'measure_name']
)['score'].mean().reset_index()

# Display sample results
measure_trends.head()
```

```
[ ]:      year      measure_name      score
0  2020  Death rate for CABG surgery patients  0.545323
1  2020      Death rate for COPD patients  5.232392
2  2020  Death rate for heart attack patients  5.281440
3  2020  Death rate for heart failure patients  7.847772
4  2020      Death rate for pneumonia patients 13.692237
```

```
[ ]: # Top complication measures to visualize
top_measure_names = top_measures.index.tolist()[:5]

# Filter trend data
top_measure_trends = measure_trends[
    measure_trends['measure_name'].isin(top_measure_names)
]

# Plot trends
plt.figure(figsize=(12,6))

for measure in top_measure_names:
    measure_data = top_measure_trends[
        top_measure_trends['measure_name'] == measure
    ]

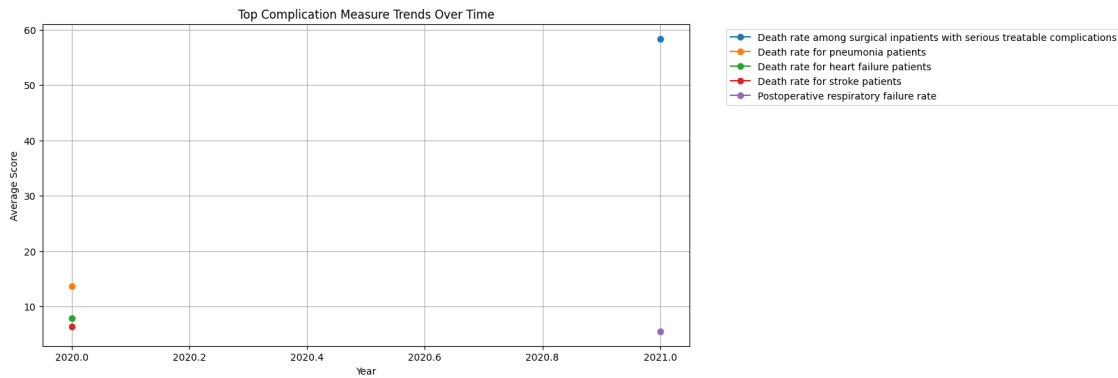
    plt.plot(
        measure_data['year'],
        measure_data['score'],
        marker='o',
        label=measure
    )

plt.title('Top Complication Measure Trends Over Time')
plt.xlabel('Year')
plt.ylabel('Average Score')

plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

plt.grid(True)

plt.show()
```



7 Patient Satisfaction Analysis

Patient satisfaction is an important healthcare quality metric.

This section analyzes HCAHPS survey data to evaluate: - Nurse communication - Hospital satisfaction trends - Top and bottom-performing hospitals - State-level satisfaction differences

```
[ ]: # Load HCAHPS dataset
hcahps_df = pd.read_csv(
    '/content/hcahps_hospital.csv',
    low_memory=False
)

# Display basic info
print("Shape:", hcahps_df.shape)

hcahps_df.info()
```

Shape: (444447, 22)

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 444447 entries, 0 to 444446

Data columns (total 22 columns):

#	Column	Non-Null Count	Dtype
0	facility_id	429195 non-null	float64
1	facility_name	444447 non-null	object
2	address	444447 non-null	object
3	citytown	444447 non-null	object
4	state	444447 non-null	object
5	zip_code	444447 non-null	int64
6	countyparish	444447 non-null	object
7	telephone_number	444447 non-null	object
8	hcahps_measure_id	444447 non-null	object
9	hcahps_question	444447 non-null	object

```

10 hcahps_answer_description      444447 non-null object
11 patient_survey_star_rating    444447 non-null object
12 patient_survey_star_rating_fn 17292 non-null object
13 hcahps_answer_percent         275110 non-null float64
14 hcahps_answer_percent_footnote 115543 non-null float64
15 hcahps_linear_mean_value      444447 non-null object
16 number_of_completed_surveys    374790 non-null float64
17 number_of_completed_surveys_fn 148521 non-null float64
18 survey_response_rate_percent   374790 non-null float64
19 survey_response_rate_percent_fn 148521 non-null float64
20 start_date                    444447 non-null object
21 end_date                      444447 non-null object
dtypes: float64(7), int64(1), object(14)
memory usage: 74.6+ MB

```

```
[ ]: # Check missing values
hcahps_df.isnull().sum()
```

```

[ ]: facility_id      15252
facility_name        0
address              0
citytown             0
state                0
zip_code             0
countyparish         0
telephone_number     0
hcahps_measure_id    0
hcahps_question      0
hcahps_answer_description 0
patient_survey_star_rating 0
patient_survey_star_rating_fn 427155
hcahps_answer_percent 169337
hcahps_answer_percent_footnote 328904
hcahps_linear_mean_value 0
number_of_completed_surveys 69657
number_of_completed_surveys_fn 295926
survey_response_rate_percent 69657
survey_response_rate_percent_fn 295926
start_date           0
end_date             0
dtype: int64

```

```

[ ]: # Convert dates
hcahps_df['start_date'] = pd.to_datetime(hcahps_df['start_date'])
hcahps_df['end_date'] = pd.to_datetime(hcahps_df['end_date'])

# Convert numeric columns

```



```

numeric_cols = [
    'hcahps_answer_percent',
    'number_of_completed_surveys',
    'survey_response_rate_percent'
]

for col in numeric_cols:
    hcahps_df[col] = pd.to_numeric(hcahps_df[col], errors='coerce')

# Drop columns with excessive missing values
drop_cols = [
    'patient_survey_star_rating_fn',
    'hcahps_answer_percent_footnote',
    'number_of_completed_surveys_fn',
    'survey_response_rate_percent_fn'
]

hcahps_cleaned = hcahps_df.drop(columns=drop_cols)

# Verify cleaned dataframe
hcahps_cleaned.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 444447 entries, 0 to 444446
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   facility_id                          429195 non-null  float64
1   facility_name                        444447 non-null  object
2   address                             444447 non-null  object
3   citytown                            444447 non-null  object
4   state                               444447 non-null  object
5   zip_code                            444447 non-null  int64
6   countyparish                        444447 non-null  object
7   telephone_number                   444447 non-null  object
8   hcahps_measure_id                   444447 non-null  object
9   hcahps_question                     444447 non-null  object
10  hcahps_answer_description             444447 non-null  object
11  patient_survey_star_rating           444447 non-null  object
12  hcahps_answer_percent                 275110 non-null  float64
13  hcahps_linear_mean_value             444447 non-null  object
14  number_of_completed_surveys          374790 non-null  float64
15  survey_response_rate_percent         374790 non-null  float64
16  start_date                           444447 non-null  datetime64[ns]
17  end_date                             444447 non-null  datetime64[ns]
dtypes: datetime64[ns](2), float64(4), int64(1), object(11)
memory usage: 61.0+ MB

```

```
[ ]: # Average patient satisfaction by state

state_satisfaction = (
    hcahps_cleaned.groupby('state')['hcahps_linear_mean_value']
    .count()
    .sort_values(ascending=False)
)

# Display top states by response volume
state_satisfaction.head(10)
```

```
[ ]: state
TX      36735
CA      31620
FL      17763
IL      16461
OH      15252
PA      15159
NY      15159
GA      12462
MI      12369
KS      12090
Name: hcahps_linear_mean_value, dtype: int64
```

```
[ ]: # Average patient satisfaction score by state

avg_state_scores = (
    hcahps_cleaned.groupby('state')['hcahps_answer_percent']
    .mean()
    .sort_values(ascending=False)
)

# Top 10 highest satisfaction states
top_states = avg_state_scores.head(10)

# Bottom 10 lowest satisfaction states
bottom_states = avg_state_scores.tail(10)

top_states, bottom_states
```

```
[ ]: (state
AK      34.722222
AZ      34.722222
MD      34.722222
DC      34.722222
DE      34.722222
MA      34.722222
```

```

PR    34.722222
VI    34.722222
VT    34.722222
RI    34.722222
Name: hcahps_answer_percent, dtype: float64,
state
MS    34.698126
IA    34.694555
WY    34.693878
NE    34.693878
KS    34.687275
SD    34.682081
ND    34.660767
AS          NaN
GU          NaN
MP          NaN
Name: hcahps_answer_percent, dtype: float64)

```

```

[ ]: # Find unique survey questions
hcahps_cleaned['hcahps_question'].unique()[:20]

```

```

[ ]: array(['Patients who reported that their nurses "Always" communicated well',
          'Patients who reported that their nurses "Sometimes" or "Never"
communicated well',
          'Patients who reported that their nurses "Usually" communicated well',
          'Nurse communication - linear mean score',
          'Nurse communication - star rating',
          'Patients who reported that their nurses "Always" treated them with
courtesy and respect',
          'Patients who reported that their nurses "Sometimes" or "Never" treated
them with courtesy and respect',
          'Patients who reported that their nurses "Usually" treated them with
courtesy and respect',
          'Patients who reported that their nurses "Always" listened carefully to
them',
          'Patients who reported that their nurses "Sometimes" or "Never" listened
carefully to them',
          'Patients who reported that their nurses "Usually" listened carefully to
them',
          'Patients who reported that their nurses "Always" explained things in a
way they could understand',
          'Patients who reported that their nurses "Sometimes" or "Never" explained
things in a way they could understand',
          'Patients who reported that their nurses "Usually" explained things in a
way they could understand',
          'Patients who reported that their doctors "Always" communicated well',
          'Patients who reported that their doctors "Sometimes" or "Never"

```

```
communicated well',
    'Patients who reported that their doctors "Usually" communicated well',
    'Doctor communication - linear mean score',
    'Doctor communication - star rating',
    'Patients who reported that their doctors "Always" treated them with
courtesy and respect'],
    dtype=object)
```

```
[ ]: # Filter for nurse communication satisfaction question
```

```
nurse_comm_df = hcahps_cleaned[
    hcahps_cleaned['hcahps_question'] ==
    'Patients who reported that their nurses "Always" communicated well'
]
```

```
# Average satisfaction by state
```

```
nurse_state_scores = (
    nurse_comm_df.groupby('state')['hcahps_answer_percent']
    .mean()
    .sort_values(ascending=False)
)
```

```
# Top and bottom states
```

```
top_nurse_states = nurse_state_scores.head(10)
bottom_nurse_states = nurse_state_scores.tail(10)

top_nurse_states, bottom_nurse_states
```

```
[ ]: (state
      NE      83.882353
      LA      83.673684
      MN      83.168224
      WI      82.764228
      SD      82.666667
      KS      82.468085
      MT      82.275862
      IA      82.193878
      ME      81.970588
      NH      81.923077
      Name: hcahps_answer_percent, dtype: float64,
      state
      MD      74.930233
      FL      74.829545
      CA      74.467742
      NV      74.400000
      VI      72.000000
      PR      68.545455)
```

```

DC      68.428571
AS      NaN
GU      NaN
MP      NaN
Name: hcahps_answer_percent, dtype: float64

```

```

[ ]: # Visualize top and bottom nurse communication states

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

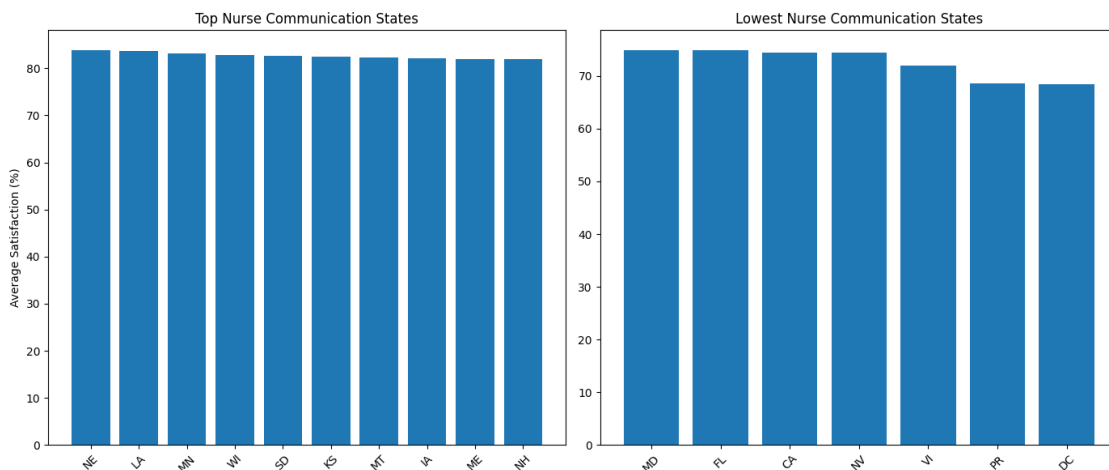
# Top states
axes[0].bar(top_nurse_states.index, top_nurse_states.values)
axes[0].set_title('Top Nurse Communication States')
axes[0].set_ylabel('Average Satisfaction (%)')
axes[0].tick_params(axis='x', rotation=45)

# Bottom states
bottom_clean = bottom_nurse_states.dropna()

axes[1].bar(bottom_clean.index, bottom_clean.values)
axes[1].set_title('Lowest Nurse Communication States')
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

```



```

[ ]: # Average nurse communication score by hospital

hospital_nurse_scores = (
    nurse_comm_df.groupby('facility_name')['hcahps_answer_percent']
    .mean()
)

```

```

        .sort_values(ascending=False)
    )

    # Top and bottom hospitals
    top_hospitals = hospital_nurse_scores.head(10)
    bottom_hospitals = hospital_nurse_scores.tail(10)

    top_hospitals, bottom_hospitals

```

```

[ ]: (facility_name
      TOWNER COUNTY MEDICAL CENTER          99.0
      SCHEURER HOSPITAL                     99.0
      ST JAMES PARISH HOSPITAL              98.0
      UNITY MEDICAL CENTER                 98.0
      EDGERTON HOSPITAL AND HEALTH SERVICES 98.0
      ERIE VA MEDICAL CENTER               97.0
      LAKE VIEW MEMORIAL HOSPITAL          97.0
      FRANKLIN GENERAL HOSPITAL            97.0
      TROUSDALE MEDICAL CENTER             97.0
      ALEXANDRIA VA MEDICAL CENTER         97.0
      Name: hcahps_answer_percent, dtype: float64,
      facility_name
      WEST VALLEY HOSPITAL                  NaN
      WESTON COUNTY HEALTH SERVICES        NaN
      WHEATLAND MEMORIAL HOSPITAL          NaN
      WHITFIELD MEDICAL SURGICAL HOSPITAL  NaN
      WICHITA COUNTY HEALTH CENTER         NaN
      WILMINGTON TREATMENT CENTER          NaN
      WILSON N JONES REGIONAL MEDICAL CENTER NaN
      WINKLER COUNTY MEMORIAL HOSPITAL     NaN
      WOODS AT PARKSIDE,THE                NaN
      YUMA DISTRICT HOSPITAL              NaN
      Name: hcahps_answer_percent, dtype: float64)

```

```

[ ]: # Remove hospitals with missing satisfaction values

hospital_nurse_scores_clean = hospital_nurse_scores.dropna()

# Recalculate rankings
top_hospitals_clean = hospital_nurse_scores_clean.head(10)
bottom_hospitals_clean = hospital_nurse_scores_clean.tail(10)

top_hospitals_clean, bottom_hospitals_clean

```

```

[ ]: (facility_name
      TOWNER COUNTY MEDICAL CENTER          99.0
      SCHEURER HOSPITAL                     99.0

```

ST JAMES PARISH HOSPITAL	98.0
UNITY MEDICAL CENTER	98.0
EDGERTON HOSPITAL AND HEALTH SERVICES	98.0
ERIE VA MEDICAL CENTER	97.0
LAKE VIEW MEMORIAL HOSPITAL	97.0
FRANKLIN GENERAL HOSPITAL	97.0
TROUSDALE MEDICAL CENTER	97.0
ALEXANDRIA VA MEDICAL CENTER	97.0

Name: hcahps_answer_percent, dtype: float64,
facility_name

BAYAMON MEDICAL CENTER	58.0
LOS ANGELES COMMUNITY HOSPITAL	57.0
DELRAY MEDICAL CENTER	56.0
MEMORIAL HOSPITAL OF GARDENA	56.0
JACKSON PARK HOSPITAL	54.0
COMMUNITY FIRST MEDICAL CENTER	52.0
COLUSA MEDICAL CENTER	52.0
COAST PLAZA HOSPITAL	51.0
LORETTO HOSPITAL	49.0
GLENN MEDICAL CENTER	48.0

Name: hcahps_answer_percent, dtype: float64)

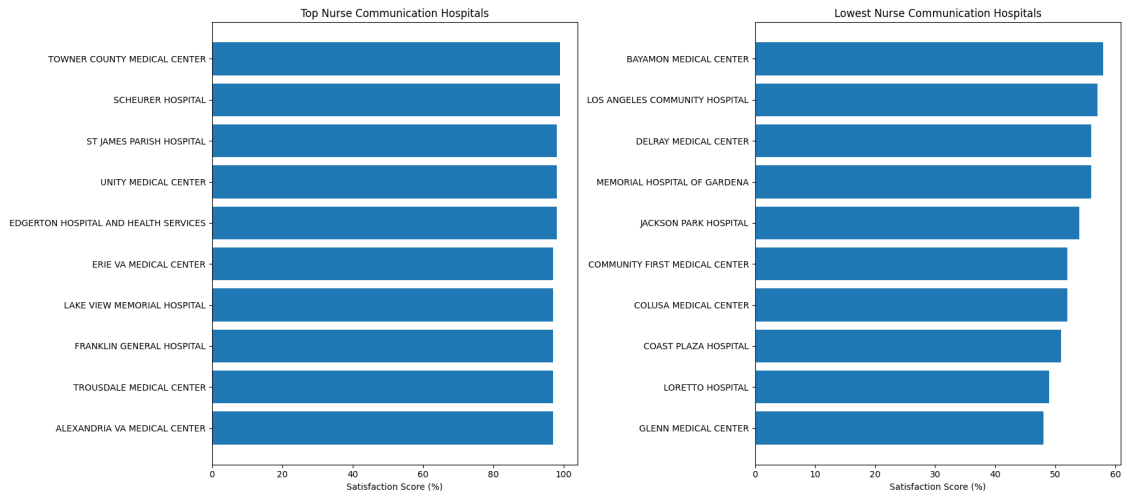
```
[ ]: import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(18, 8))

# Top hospitals
axes[0].barh(top_hospitals_clean.index, top_hospitals_clean.values)
axes[0].set_title('Top Nurse Communication Hospitals')
axes[0].set_xlabel('Satisfaction Score (%)')
axes[0].invert_yaxis()

# Bottom hospitals
axes[1].barh(bottom_hospitals_clean.index, bottom_hospitals_clean.values)
axes[1].set_title('Lowest Nurse Communication Hospitals')
axes[1].set_xlabel('Satisfaction Score (%)')
axes[1].invert_yaxis()

plt.tight_layout()
plt.show()
```



```
[ ]: # Load Medicare spending dataset
spending_df = pd.read_csv('/content/
↳medicare_hospital_spending_per_patient_hospital.csv')

# Preview dataset
spending_df.head()
```

```
[ ]: facility_id      facility_name      address \
0      10001  SOUTHEAST HEALTH MEDICAL CENTER      1108 ROSS CLARK CIRCLE
1      10005      MARSHALL MEDICAL CENTERS      2505 U S HIGHWAY 431 NORTH
2      10006      NORTH ALABAMA MEDICAL CENTER      1701 VETERANS DRIVE
3      10007      MIZELL MEMORIAL HOSPITAL      702 N MAIN ST
4      10008      CRENSHAW COMMUNITY HOSPITAL      101 HOSPITAL CIRCLE

      citytown state  zip_code  countyparish  telephone_number  measure_id \
0      DOTHAN   AL    36301    HOUSTON    (334) 793-8701      MSPB-1
1      BOAZ    AL    35957    MARSHALL    (256) 593-8310      MSPB-1
2  FLORENCE    AL    35630    LAUDERDALE    (256) 629-1000      MSPB-1
3      OPP     AL    36467    COVINGTON    (334) 493-3541      MSPB-1
4  LUVERNE    AL    36049    CRENSHAW    (334) 335-3374      MSPB-1

      measure_name  score  footnote \
0  Medicare hospital spending per patient (Medica...  1.04      NaN
1  Medicare hospital spending per patient (Medica...  0.95      NaN
2  Medicare hospital spending per patient (Medica...  1.00      NaN
3  Medicare hospital spending per patient (Medica...  1.01      NaN
4  Medicare hospital spending per patient (Medica...  1.11      NaN

      start_date  end_date
0  2022-01-01  2022-12-31
```



```

1 2022-01-01 2022-12-31
2 2022-01-01 2022-12-31
3 2022-01-01 2022-12-31
4 2022-01-01 2022-12-31

```

```
[ ]: # Dataset structure
      spending_df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4615 entries, 0 to 4614
Data columns (total 14 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   facility_id           4615 non-null   int64
 1   facility_name         4615 non-null   object
 2   address               4615 non-null   object
 3   citytown              4615 non-null   object
 4   state                 4615 non-null   object
 5   zip_code              4615 non-null   int64
 6   countyparish          4615 non-null   object
 7   telephone_number     4615 non-null   object
 8   measure_id            4615 non-null   object
 9   measure_name          4615 non-null   object
10   score                 2939 non-null   float64
11   footnote              1676 non-null   float64
12   start_date            4615 non-null   object
13   end_date              4615 non-null   object
dtypes: float64(2), int64(2), object(10)
memory usage: 504.9+ KB

```

```
[ ]: # Check missing values
      spending_df.isnull().sum()
```

```

[ ]: facility_id           0
      facility_name       0
      address             0
      citytown            0
      state               0
      zip_code            0
      countyparish        0
      telephone_number    0
      measure_id          0
      measure_name        0
      score               1676
      footnote            2939
      start_date          0
      end_date            0

```

dtype: int64

```
[ ]: # Remove rows with missing spending scores
spending_cleaned = spending_df.dropna(subset=['score'])

# Verify cleaning
spending_cleaned.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 2939 entries, 0 to 4607
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   facility_id           2939 non-null   int64
1   facility_name         2939 non-null   object
2   address               2939 non-null   object
3   citytown              2939 non-null   object
4   state                 2939 non-null   object
5   zip_code              2939 non-null   int64
6   countyparish          2939 non-null   object
7   telephone_number     2939 non-null   object
8   measure_id           2939 non-null   object
9   measure_name          2939 non-null   object
10  score                 2939 non-null   float64
11  footnote              0 non-null      float64
12  start_date            2939 non-null   object
13  end_date              2939 non-null   object
dtypes: float64(2), int64(2), object(10)
memory usage: 344.4+ KB
```

```
[ ]: # Average spending by state
state_spending = spending_cleaned.groupby('state')['score'].mean().
    ↪sort_values(ascending=False)

# Top and bottom states
top_spending_states = state_spending.head(10)
bottom_spending_states = state_spending.tail(10)

top_spending_states, bottom_spending_states
```

```
[ ]: (state
LA    1.077403
NJ    1.068197
TX    1.041679
NV    1.034000
AR    1.022195
DE    1.021667
```

```

MS      1.018846
IN      1.014750
NY      1.013780
TN      1.011818
Name: score, dtype: float64,
state
UT      0.935312
WA      0.929778
VT      0.928333
WI      0.926562
MN      0.919556
ND      0.917143
HI      0.907500
MT      0.904545
OR      0.893030
AK      0.876250
Name: score, dtype: float64)

```

8 Nurse Communication Performance

Communication quality strongly impacts patient satisfaction and healthcare outcomes.

This section identifies the hospitals and states with the strongest and weakest nurse communication scores.

```

[ ]: # Nurse communication satisfaction by hospital
hospital_satisfaction = hcahps_cleaned[
    hcahps_cleaned['hcahps_question'].str.contains(
        'nurses "Always" communicated well',
        case=False,
        na=False
    )
]

# Average satisfaction per hospital
hospital_satisfaction_scores = hospital_satisfaction.groupby(
    'facility_id'
)['hcahps_answer_percent'].mean().reset_index()

# Rename column
hospital_satisfaction_scores.columns = [
    'facility_id',
    'satisfaction_score'
]

hospital_satisfaction_scores.head()

```

```
[ ]: facility_id  satisfaction_score
0      10001.0             72.0
1      10005.0             78.0
2      10006.0             73.0
3      10007.0             80.0
4      10008.0             85.0
```

```
[ ]: # Average spending per hospital
hospital_spending_scores = spending_cleaned.groupby(
    'facility_id'
)['score'].mean().reset_index()

# Rename column
hospital_spending_scores.columns = [
    'facility_id',
    'spending_score'
]

hospital_spending_scores.head()
```

```
[ ]: facility_id  spending_score
0      10001             1.04
1      10005             0.95
2      10006             1.00
3      10007             1.01
4      10008             1.11
```

```
[ ]: # Merge datasets
merged_scores = pd.merge(
    hospital_satisfaction_scores,
    hospital_spending_scores,
    on='facility_id',
    how='inner'
)

merged_scores.head()
```

```
[ ]: facility_id  satisfaction_score  spending_score
0      10001.0             72.0             1.04
1      10005.0             78.0             0.95
2      10006.0             73.0             1.00
3      10007.0             80.0             1.01
4      10008.0             85.0             1.11
```

```
[ ]: # Correlation between spending and satisfaction
correlation = merged_scores[
    ['satisfaction_score', 'spending_score']
```

```
].corr()

correlation
```

```
[ ]:          satisfaction_score  spending_score
satisfaction_score          1.000000        -0.232713
spending_score             -0.232713         1.000000
```

```
[ ]: import matplotlib.pyplot as plt

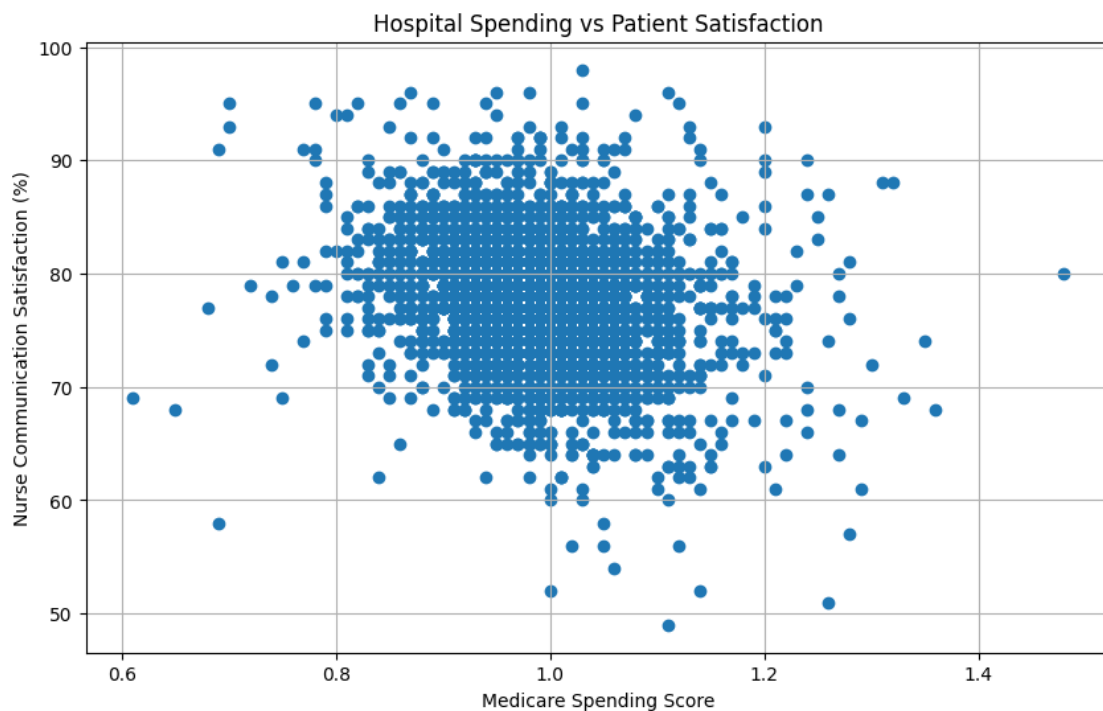
# Scatter plot
plt.figure(figsize=(10,6))

plt.scatter(
    merged_scores['spending_score'],
    merged_scores['satisfaction_score']
)

plt.title('Hospital Spending vs Patient Satisfaction')
plt.xlabel('Medicare Spending Score')
plt.ylabel('Nurse Communication Satisfaction (%)')

plt.grid(True)

plt.show()
```



```
[ ]: # Remove any remaining invalid rows
merged_scores_clean = merged_scores.dropna()

# Verify
merged_scores_clean.isnull().sum()
```

```
[ ]: facility_id      0
satisfaction_score   0
spending_score       0
dtype: int64
```

```
[ ]: import numpy as np
import matplotlib.pyplot as plt

x = merged_scores_clean['spending_score']
y = merged_scores_clean['satisfaction_score']

# Regression calculation
m, b = np.polyfit(x, y, 1)

# Create regression line
x_line = np.linspace(x.min(), x.max(), 100)
y_line = m * x_line + b

plt.figure(figsize=(10,6))

# Scatter plot
plt.scatter(
    x,
    y,
    alpha=0.2,
    s=20
)

# Regression line
plt.plot(
    x_line,
    y_line,
    color='red',
    linewidth=5,
    zorder=10,
    label='Regression Line'
)

plt.title('Hospital Spending vs Patient Satisfaction')
plt.xlabel('Medicare Spending Score')
```

```
plt.ylabel('Nurse Communication Satisfaction (%)')

plt.legend()
plt.grid(True)

plt.show()

print("Slope:", m)
print("Intercept:", b)
```



```
Slope: -16.92119479937701
Intercept: 94.23886127788731
```

```
[ ]: complications_df.head()
```

```
[ ]:
  facility_id      facility_name      address \
0    10001.0  SOUTHEAST HEALTH MEDICAL CENTER  1108 ROSS CLARK CIRCLE
1    10001.0  SOUTHEAST HEALTH MEDICAL CENTER  1108 ROSS CLARK CIRCLE
2    10001.0  SOUTHEAST HEALTH MEDICAL CENTER  1108 ROSS CLARK CIRCLE
3    10001.0  SOUTHEAST HEALTH MEDICAL CENTER  1108 ROSS CLARK CIRCLE
4    10001.0  SOUTHEAST HEALTH MEDICAL CENTER  1108 ROSS CLARK CIRCLE

  citytown state  zip_code  countyparish  telephone_number  measure_id \
0  DOTHAN   AL    36301    HOUSTON    (334) 793-8701  COMP_HIP_KNEE
```

1	DOTHAN	AL	36301	HOUSTON	(334) 793-8701	MORT_30_AMI
2	DOTHAN	AL	36301	HOUSTON	(334) 793-8701	MORT_30_CABG
3	DOTHAN	AL	36301	HOUSTON	(334) 793-8701	MORT_30_COPD
4	DOTHAN	AL	36301	HOUSTON	(334) 793-8701	MORT_30_HF

	measure_name	\
0	Rate of complications for hip/knee replacement...	
1	Death rate for heart attack patients	
2	Death rate for CABG surgery patients	
3	Death rate for COPD patients	
4	Death rate for heart failure patients	

	compared_to_national	denominator	score	lower_estimate	\
0	No Different Than the National Rate	32.0	3.0	1.6	
1	No Different Than the National Rate	291.0	10.8	8.4	
2	No Different Than the National Rate	157.0	3.8	2.1	
3	No Different Than the National Rate	122.0	7.9	5.2	
4	No Different Than the National Rate	610.0	10.5	8.6	

	higher_estimate	footnote	start_date	end_date
0	5.5	NaN	2020-07-01	2023-03-31
1	13.6	NaN	2020-07-01	2023-06-30
2	6.8	NaN	2020-07-01	2023-06-30
3	11.9	NaN	2020-07-01	2023-06-30
4	12.9	NaN	2020-07-01	2023-06-30

```
[ ]: complications_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 90801 entries, 0 to 90800
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   facility_id                          87685 non-null  float64
1   facility_name                        90801 non-null  object
2   address                             90801 non-null  object
3   citytown                            90801 non-null  object
4   state                               90801 non-null  object
5   zip_code                            90801 non-null  int64
6   countyparish                        90801 non-null  object
7   telephone_number                   90801 non-null  object
8   measure_id                          90801 non-null  object
9   measure_name                        90801 non-null  object
10  compared_to_national                 90801 non-null  object
11  denominator                          90801 non-null  float64
12  score                               90801 non-null  float64
13  lower_estimate                       90801 non-null  float64
```



```

14 higher_estimate      90801 non-null float64
15 footnote             42306 non-null float64
16 start_date           90801 non-null datetime64[ns]
17 end_date             90801 non-null datetime64[ns]
dtypes: datetime64[ns](2), float64(6), int64(1), object(9)
memory usage: 12.5+ MB

```

```
[ ]: complications_df.columns
```

```
[ ]: Index(['facility_id', 'facility_name', 'address', 'citytown', 'state',
           'zip_code', 'countyparish', 'telephone_number', 'measure_id',
           'measure_name', 'compared_to_national', 'denominator', 'score',
           'lower_estimate', 'higher_estimate', 'footnote', 'start_date',
           'end_date'],
          dtype='object')
```

```
[ ]: timely_df.head()
```

```
[ ]:
  facility_id      facility_name      address \
0    10001.0  SOUTHEAST HEALTH MEDICAL CENTER  1108 ROSS CLARK CIRCLE
1    10001.0  SOUTHEAST HEALTH MEDICAL CENTER  1108 ROSS CLARK CIRCLE
2    10001.0  SOUTHEAST HEALTH MEDICAL CENTER  1108 ROSS CLARK CIRCLE
3    10001.0  SOUTHEAST HEALTH MEDICAL CENTER  1108 ROSS CLARK CIRCLE
4    10001.0  SOUTHEAST HEALTH MEDICAL CENTER  1108 ROSS CLARK CIRCLE

  citytown state  zip_code countyparish telephone_number \
0  DOTHAN   AL    36301    HOUSTON    (334) 793-8701
1  DOTHAN   AL    36301    HOUSTON    (334) 793-8701
2  DOTHAN   AL    36301    HOUSTON    (334) 793-8701
3  DOTHAN   AL    36301    HOUSTON    (334) 793-8701
4  DOTHAN   AL    36301    HOUSTON    (334) 793-8701

              condition      measure_id \
0      Emergency Department          EDV
1  Electronic Clinical Quality Measure  ED_2_Strata_1
2  Electronic Clinical Quality Measure  ED_2_Strata_2
3      Healthcare Personnel Vaccination  HCP_COVID_19
4  Electronic Clinical Quality Measure      HH_01

              measure_name  score  sample  footnote \
0      Emergency department volume    NaN    NaN    NaN
1  Admit Decision Time to ED Departure Time for A...    NaN    NaN    5.0
2  Admit Decision Time to ED Departure Time for A...    NaN    NaN    5.0
3  Percentage of healthcare personnel who are up ...    0.0  2712.0    NaN
4      Hospital Harm - Severe Hypoglycemia    NaN    NaN    5.0

  start_date  end_date
```

```

0  2022-01-01  2022-12-31
1  2023-01-01  2023-12-31
2  2023-01-01  2023-12-31
3  2023-10-01  2023-12-31
4  2023-01-01  2023-12-31

```

```
[ ]: timely_df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 118520 entries, 0 to 118519
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   facility_id           118072 non-null  float64
1   facility_name         118520 non-null  object
2   address               118520 non-null  object
3   citytown              118520 non-null  object
4   state                 118520 non-null  object
5   zip_code              118520 non-null  int64
6   countyparish          118520 non-null  object
7   telephone_number     118520 non-null  object
8   condition             118520 non-null  object
9   measure_id            118520 non-null  object
10  measure_name          118520 non-null  object
11  score                 50726 non-null   float64
12  sample                51697 non-null   float64
13  footnote              71660 non-null   float64
14  start_date            118520 non-null  object
15  end_date              118520 non-null  object
dtypes: float64(4), int64(1), object(11)
memory usage: 14.5+ MB

```

```
[ ]: timely_df.columns
```

```

[ ]: Index(['facility_id', 'facility_name', 'address', 'citytown', 'state',
          'zip_code', 'countyparish', 'telephone_number', 'condition',
          'measure_id', 'measure_name', 'score', 'sample', 'footnote',
          'start_date', 'end_date'],
          dtype='object')

```

```

[ ]: state_complication_scores = (
    complications_df
    .groupby('state')['score']
    .mean()
    .sort_values(ascending=False)
    .reset_index()
)

```

```
state_complication_scores.head(10)
```

```
[ ]:   state      score
0    NJ  11.309093
1    MD  10.741469
2    DC  10.497368
3    FL   9.769033
4    CT   9.392368
5    MA   8.825279
6    NY   8.159693
7    VA   8.143533
8    SC   7.827985
9    DE   7.672105
```

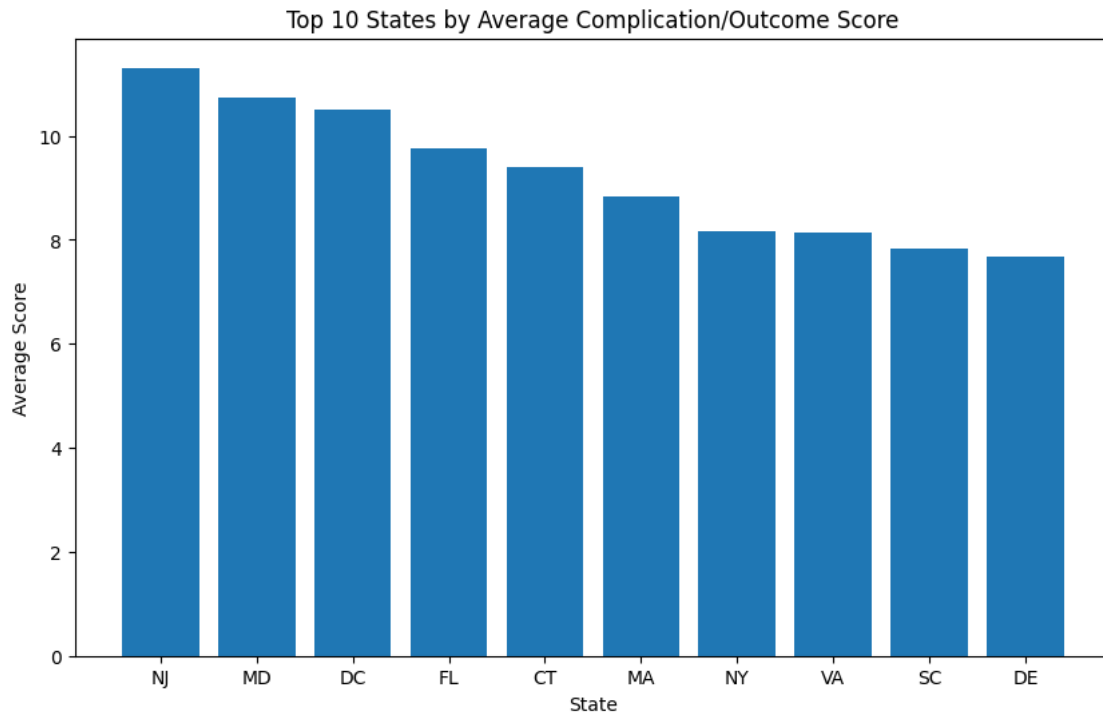
```
[ ]: import matplotlib.pyplot as plt

top_states = state_complication_scores.head(10)

plt.figure(figsize=(10,6))
plt.bar(top_states['state'], top_states['score'])

plt.title('Top 10 States by Average Complication/Outcome Score')
plt.xlabel('State')
plt.ylabel('Average Score')

plt.show()
```



```
[ ]: timely_state_scores = (
    timely_df
    .dropna(subset=['score'])
    .groupby('state')['score']
    .mean()
    .sort_values(ascending=False)
    .reset_index()
)

timely_state_scores.head(10)
```

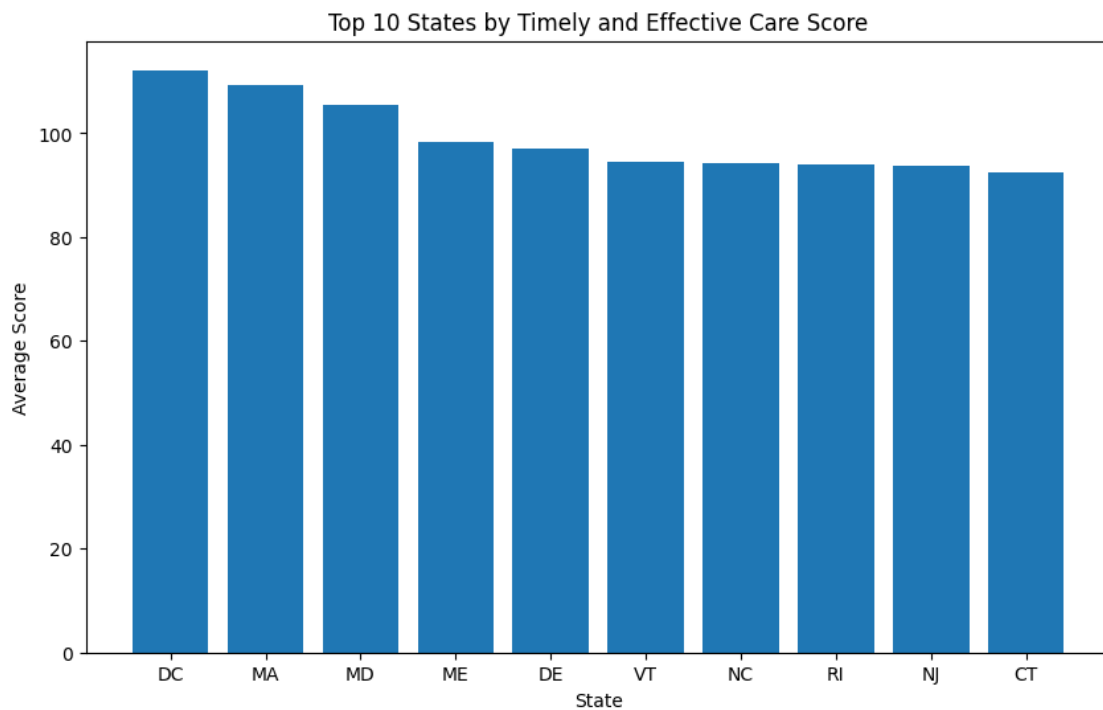
```
[ ]:   state      score
0    DC  112.154023
1    MA  109.331343
2    MD  105.432964
3    ME   98.256303
4    DE   97.100000
5    VT   94.385039
6    NC   94.126853
7    RI   94.098621
8    NJ   93.702246
9    CT   92.519685
```

```
[ ]: top_timely_states = timely_state_scores.head(10)

plt.figure(figsize=(10,6))
plt.bar(top_timely_states['state'], top_timely_states['score'])

plt.title('Top 10 States by Timely and Effective Care Score')
plt.xlabel('State')
plt.ylabel('Average Score')

plt.show()
```



9 Final Conclusions

9.1 Key Findings

- Certain states consistently showed higher complication scores.
- Patient satisfaction varied significantly across hospitals and regions.
- Some hospitals demonstrated strong communication scores despite moderate spending.
- Higher Medicare spending did not always correspond to better outcomes.

9.2 Business Impact

Healthcare organizations can use this analysis to: - Identify underperforming regions - Improve patient communication strategies - Optimize spending efficiency - Monitor healthcare quality trends

9.3 Skills Demonstrated

- Data Cleaning
- Exploratory Data Analysis
- Data Visualization
- Statistical Analysis
- Healthcare Analytics
- Python Programming
- Real-world Data Handling

```
[1]: !jupyter nbconvert --to pdf "/content/healthcare.ipynb"
```

```
[NbConvertApp] WARNING | pattern '/content/healthcare.ipynb' matched no files
This application is used to convert notebook files (*.ipynb)
to various other formats.
```

```
WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.
```

Options

=====

The options below are convenience aliases to configurable class-options, as listed in the "Equivalent to" description-line of the aliases.

To see all configurable class-options for some <cmd>, use:

```
<cmd> --help-all
```

--debug

```
set log level to logging.DEBUG (maximize logging output)
```

```
Equivalent to: [--Application.log_level=10]
```

--show-config

```
Show the application's configuration (human-readable format)
```

```
Equivalent to: [--Application.show_config=True]
```

--show-config-json

```
Show the application's configuration (json format)
```

```
Equivalent to: [--Application.show_config_json=True]
```

--generate-config

```
generate default config file
```

```
Equivalent to: [--JupyterApp.generate_config=True]
```

-y

```
Answer yes to any questions instead of prompting.
```

```
Equivalent to: [--JupyterApp.answer_yes=True]
```

--execute

```
Execute the notebook prior to export.
```

Equivalent to: [--ExecutePreprocessor.enabled=True]

--allow-errors

Continue notebook execution even if one of the cells throws an error and include the error message in the cell output (the default behaviour is to abort conversion). This flag is only relevant if '--execute' was specified, too.

Equivalent to: [--ExecutePreprocessor.allow_errors=True]

--stdin

read a single notebook file from stdin. Write the resulting notebook with default basename 'notebook.*'

Equivalent to: [--NbConvertApp.from_stdin=True]

--stdout

Write notebook output to stdout instead of files.

Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]

--inplace

Run nbconvert in place, overwriting the existing notebook (only relevant when converting to notebook format)

Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=]

--clear-output

Clear output of current file and save in place, overwriting the existing notebook.

Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=
--ClearOutputPreprocessor.enabled=True]

--coalesce-streams

Coalesce consecutive stdout and stderr outputs into one stream (within each cell).

Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=
--CoalesceStreamsPreprocessor.enabled=True]

--no-prompt

Exclude input and output prompts from converted document.

Equivalent to: [--TemplateExporter.exclude_input_prompt=True
--TemplateExporter.exclude_output_prompt=True]

--no-input

Exclude input cells and output prompts from converted document.

This mode is ideal for generating code-free reports.

Equivalent to: [--TemplateExporter.exclude_output_prompt=True
--TemplateExporter.exclude_input=True
--TemplateExporter.exclude_input_prompt=True]

--allow-chromium-download

Whether to allow downloading chromium if no suitable version is found on the system.

Equivalent to: [--WebPDFExporter.allow_chromium_download=True]

--disable-chromium-sandbox

Disable chromium security sandbox when converting to PDF..

Equivalent to: [--WebPDFExporter.disable_sandbox=True]

--show-input

Shows code input. This flag is only useful for dejavu users.
 Equivalent to: [--TemplateExporter.exclude_input=False]

--embed-images
 Embed the images as base64 dataurls in the output. This flag is only useful for the HTML/WebPDF/Slides exports.
 Equivalent to: [--HTMLExporter.embed_images=True]

--sanitize-html
 Whether the HTML in Markdown cells and cell outputs should be sanitized..
 Equivalent to: [--HTMLExporter.sanitize_html=True]

--log-level=<Enum>
 Set the log level by value or name.
 Choices: any of [0, 10, 20, 30, 40, 50, 'DEBUG', 'INFO', 'WARN', 'ERROR', 'CRITICAL']
 Default: 30
 Equivalent to: [--Application.log_level]

--config=<Unicode>
 Full path of a config file.
 Default: ''
 Equivalent to: [--JupyterApp.config_file]

--to=<Unicode>
 The export format to be used, either one of the built-in formats
 ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf']
 or a dotted object name that represents the import path for an
 ``Exporter`` class
 Default: ''
 Equivalent to: [--NbConvertApp.export_format]

--template=<Unicode>
 Name of the template to use
 Default: ''
 Equivalent to: [--TemplateExporter.template_name]

--template-file=<Unicode>
 Name of the template file to use
 Default: None
 Equivalent to: [--TemplateExporter.template_file]

--theme=<Unicode>
 Template specific theme(e.g. the name of a JupyterLab CSS theme distributed as prebuilt extension for the lab template)
 Default: 'light'
 Equivalent to: [--HTMLExporter.theme]

--sanitize_html=<Bool>
 Whether the HTML in Markdown cells and cell outputs should be sanitized.This should be set to True by nbviewer or similar tools.
 Default: False
 Equivalent to: [--HTMLExporter.sanitize_html]

--writer=<DottedObjectName>
 Writer class used to write the
 results of the conversion


```
> jupyter nbconvert mynotebook.ipynb --to html
```

Options include ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf'].

```
> jupyter nbconvert --to latex mynotebook.ipynb
```

Both HTML and LaTeX support multiple output templates. LaTeX includes

'base', 'article' and 'report'. HTML includes 'basic', 'lab' and 'classic'. You can specify the flavor of the format used.

```
> jupyter nbconvert --to html --template lab mynotebook.ipynb
```

You can also pipe the output to stdout, rather than a file

```
> jupyter nbconvert mynotebook.ipynb --stdout
```

PDF is generated via latex

```
> jupyter nbconvert mynotebook.ipynb --to pdf
```

You can get (and serve) a Reveal.js-powered slideshow

```
> jupyter nbconvert myslides.ipynb --to slides --post serve
```

Multiple notebooks can be given at the command line in a couple of different ways:

```
> jupyter nbconvert notebook*.ipynb
```

```
> jupyter nbconvert notebook1.ipynb notebook2.ipynb
```

or you can specify the notebooks list in a config file, containing::

```
c.NbConvertApp.notebooks = ["my_notebook.ipynb"]
```

```
> jupyter nbconvert --config mycfg.py
```

To see all available configurables, use `--help-all`.

```
[ ]: !apt-get install texlive-xetex texlive-fonts-recommended texlive-plain-generic
↪-y
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

The following additional packages will be installed:

dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java
libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12
libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0
libsynctex2 libteckit0 libtexlua53 libtexluajit2 libwoff1 libzip-0-13
lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration t1utils
teckit tex-common tex-gyre texlive-base texlive-binaries texlive-latex-base
texlive-latex-extra texlive-latex-recommended texlive-pictures tipa
xfonts-encodings xfonts-utils

Suggested packages:

fonts-noto fonts-freefont-otf | fonts-freefont-ttf libavalon-framework-java
libcommons-logging-java-doc libexcalibur-logkit-java liblog4j1.2-java
poppler-utils ghostscript fonts-japanese-mincho | fonts-ipafont-mincho
fonts-japanese-gothic | fonts-ipafont-gothic fonts-arphic-ukai
fonts-arphic-uming fonts-nanum ri ruby-dev bundler debhelper gv
| postscript-viewer perl-tk xpdf | pdf-viewer xzdec
texlive-fonts-recommended-doc texlive-latex-base-doc python3-pygments
icc-profiles libfile-which-perl libspreadsheet-parseexcel-perl
texlive-latex-extra-doc texlive-latex-recommended-doc texlive-luatex
texlive-pstricks dot2tex prerex texlive-pictures-doc vprerex
default-jre-headless tipa-doc

The following NEW packages will be installed:

dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java
libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12
libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0
libsynctex2 libteckit0 libtexlua53 libtexluajit2 libwoff1 libzip-0-13
lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration t1utils
teckit tex-common tex-gyre texlive-base texlive-binaries
texlive-fonts-recommended texlive-latex-base texlive-latex-extra
texlive-latex-recommended texlive-pictures texlive-plain-generic
texlive-xetex tipa xfonts-encodings xfonts-utils

0 upgraded, 53 newly installed, 0 to remove and 3 not upgraded.

Need to get 182 MB of archives.

After this operation, 571 MB of additional disk space will be used.

Get:1 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 fonts-droid-fallback all 1:6.0.1r16-1.1build1 [1,805 kB]

Get:2 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 fonts-lato all 2.0-2.1 [2,696 kB]

Get:3 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 poppler-data all 0.4.11-1 [2,171 kB]

Get:4 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 tex-common all 6.17 [33.7 kB]

Get:5 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 fonts-urw-base35 all 20200910-1 [6,367 kB]

Get:6 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libgs9-common
all 9.55.0~dfsg1-0ubuntu5.13 [753 kB]
Get:7 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libidn12 amd64
1.38-4ubuntu1 [60.0 kB]
Get:8 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 libijs-0.35 amd64
0.35-15build2 [16.5 kB]
Get:9 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 libjbig2dec0 amd64
0.19-3build2 [64.7 kB]
Get:10 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libgs9 amd64
9.55.0~dfsg1-0ubuntu5.13 [5,032 kB]
Get:11 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libkpathsea6
amd64 2021.20210626.59705-1ubuntu0.3 [60.6 kB]
Get:12 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 libwoff1 amd64
1.0.2-1build4 [45.2 kB]
Get:13 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 dvisvgm amd64
2.13.1-1 [1,221 kB]
Get:14 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 fonts-lmodern all
2.004.5-6.1 [4,532 kB]
Get:15 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 fonts-noto-mono all
20201225-1build1 [397 kB]
Get:16 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 fonts-texgyre all
20180621-3.1 [10.2 MB]
Get:17 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libapache-pom-java
all 18-1 [4,720 B]
Get:18 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libcommons-parent-
java all 43-1 [10.8 kB]
Get:19 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libcommons-logging-
java all 1.2-2 [60.3 kB]
Get:20 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libptexenc1
amd64 2021.20210626.59705-1ubuntu0.3 [39.1 kB]
Get:21 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 rubygems-integration
all 1.18 [5,336 B]
Get:22 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby3.0 amd64
3.0.2-7ubuntu2.12 [50.1 kB]
Get:23 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-rubygems
all 3.3.5-2ubuntu1.2 [228 kB]
Get:24 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 ruby amd64 1:3.0~exp1
[5,100 B]
Get:25 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 rake all 13.0.6-2 [61.7
kB]
Get:26 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 ruby-net-telnet all
0.1.1-2 [12.6 kB]
Get:27 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-webrick
all 1.7.0-3ubuntu0.2 [52.5 kB]
Get:28 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-xmlrpc all
0.3.2-1ubuntu0.1 [24.9 kB]
Get:29 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libruby3.0
amd64 3.0.2-7ubuntu2.12 [5,113 kB]

Get:30 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libsynctex2
 amd64 2021.20210626.59705-1ubuntu0.3 [55.8 kB]
 Get:31 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libteckit0 amd64
 2.5.11+ds1-1 [421 kB]
 Get:32 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libtexlua53
 amd64 2021.20210626.59705-1ubuntu0.3 [120 kB]
 Get:33 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libtexluajit2
 amd64 2021.20210626.59705-1ubuntu0.3 [267 kB]
 Get:34 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libzip-0-13 amd64
 0.13.72+dfsg.1-1.1 [27.0 kB]
 Get:35 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 xfonts-encodings all
 1:1.0.5-0ubuntu2 [578 kB]
 Get:36 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 xfonts-utils amd64
 1:7.7+6build2 [94.6 kB]
 Get:37 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 lmodern all
 2.004.5-6.1 [9,471 kB]
 Get:38 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 preview-latex-style
 all 12.2-1ubuntu1 [185 kB]
 Get:39 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 t1utils amd64
 1.41-4build2 [61.3 kB]
 Get:40 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 teckit amd64
 2.5.11+ds1-1 [699 kB]
 Get:41 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 tex-gyre all
 20180621-3.1 [6,209 kB]
 Get:42 <http://archive.ubuntu.com/ubuntu> jammy-updates/universe amd64 texlive-
 binaries amd64 2021.20210626.59705-1ubuntu0.3 [9,861 kB]
 Get:43 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-base all
 2021.20220204-1 [21.0 MB]
 Get:44 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-fonts-
 recommended all 2021.20220204-1 [4,972 kB]
 Get:45 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-latex-base
 all 2021.20220204-1 [1,128 kB]
 Get:46 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libfontbox-java all
 1:1.8.16-2 [207 kB]
 Get:47 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libpdfbox-java all
 1:1.8.16-2 [5,199 kB]
 Get:48 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-latex-
 recommended all 2021.20220204-1 [14.4 MB]
 Get:49 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-pictures
 all 2021.20220204-1 [8,720 kB]
 Get:50 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-latex-extra
 all 2021.20220204-1 [13.9 MB]
 Get:51 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-plain-
 generic all 2021.20220204-1 [27.5 MB]
 Get:52 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 tipa all 2:1.3-21
 [2,967 kB]
 Get:53 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-xetex all
 2021.20220204-1 [12.4 MB]

```

Fetched 182 MB in 3s (61.3 MB/s)
Extracting templates from packages: 100%
Preconfiguring packages ...
Selecting previously unselected package fonts-droid-fallback.
(Reading database ... 118243 files and directories currently installed.)
Preparing to unpack .../00-fonts-droid-fallback_1%3a6.0.1r16-1.1build1_all.deb
...
Unpacking fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Selecting previously unselected package fonts-lato.
Preparing to unpack .../01-fonts-lato_2.0-2.1_all.deb ...
Unpacking fonts-lato (2.0-2.1) ...
Selecting previously unselected package poppler-data.
Preparing to unpack .../02-poppler-data_0.4.11-1_all.deb ...
Unpacking poppler-data (0.4.11-1) ...
Selecting previously unselected package tex-common.
Preparing to unpack .../03-tex-common_6.17_all.deb ...
Unpacking tex-common (6.17) ...
Selecting previously unselected package fonts-urw-base35.
Preparing to unpack .../04-fonts-urw-base35_20200910-1_all.deb ...
Unpacking fonts-urw-base35 (20200910-1) ...
Selecting previously unselected package libgs9-common.
Preparing to unpack .../05-libgs9-common_9.55.0~dfsg1-0ubuntu5.13_all.deb ...
Unpacking libgs9-common (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libidn12:amd64.
Preparing to unpack .../06-libidn12_1.38-4ubuntu1_amd64.deb ...
Unpacking libidn12:amd64 (1.38-4ubuntu1) ...
Selecting previously unselected package libijs-0.35:amd64.
Preparing to unpack .../07-libijs-0.35_0.35-15build2_amd64.deb ...
Unpacking libijs-0.35:amd64 (0.35-15build2) ...
Selecting previously unselected package libjbig2dec0:amd64.
Preparing to unpack .../08-libjbig2dec0_0.19-3build2_amd64.deb ...
Unpacking libjbig2dec0:amd64 (0.19-3build2) ...
Selecting previously unselected package libgs9:amd64.
Preparing to unpack .../09-libgs9_9.55.0~dfsg1-0ubuntu5.13_amd64.deb ...
Unpacking libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libkpathsea6:amd64.
Preparing to unpack .../10-libkpathsea6_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libwoff1:amd64.
Preparing to unpack .../11-libwoff1_1.0.2-1build4_amd64.deb ...
Unpacking libwoff1:amd64 (1.0.2-1build4) ...
Selecting previously unselected package dvisvgm.
Preparing to unpack .../12-dvisvgm_2.13.1-1_amd64.deb ...
Unpacking dvisvgm (2.13.1-1) ...
Selecting previously unselected package fonts-lmodern.
Preparing to unpack .../13-fonts-lmodern_2.004.5-6.1_all.deb ...
Unpacking fonts-lmodern (2.004.5-6.1) ...

```

Selecting previously unselected package fonts-noto-mono.
Preparing to unpack .../14-fonts-noto-mono_20201225-1build1_all.deb ...
Unpacking fonts-noto-mono (20201225-1build1) ...
Selecting previously unselected package fonts-texgyre.
Preparing to unpack .../15-fonts-texgyre_20180621-3.1_all.deb ...
Unpacking fonts-texgyre (20180621-3.1) ...
Selecting previously unselected package libapache-pom-java.
Preparing to unpack .../16-libapache-pom-java_18-1_all.deb ...
Unpacking libapache-pom-java (18-1) ...
Selecting previously unselected package libcommons-parent-java.
Preparing to unpack .../17-libcommons-parent-java_43-1_all.deb ...
Unpacking libcommons-parent-java (43-1) ...
Selecting previously unselected package libcommons-logging-java.
Preparing to unpack .../18-libcommons-logging-java_1.2-2_all.deb ...
Unpacking libcommons-logging-java (1.2-2) ...
Selecting previously unselected package libptexenc1:amd64.
Preparing to unpack .../19-libptexenc1_2021.20210626.59705-1ubuntu0.3_amd64.deb ...
Unpacking libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package rubygems-integration.
Preparing to unpack .../20-rubygems-integration_1.18_all.deb ...
Unpacking rubygems-integration (1.18) ...
Selecting previously unselected package ruby3.0.
Preparing to unpack .../21-ruby3.0_3.0.2-7ubuntu2.12_amd64.deb ...
Unpacking ruby3.0 (3.0.2-7ubuntu2.12) ...
Selecting previously unselected package ruby-rubygems.
Preparing to unpack .../22-ruby-rubygems_3.3.5-2ubuntu1.2_all.deb ...
Unpacking ruby-rubygems (3.3.5-2ubuntu1.2) ...
Selecting previously unselected package ruby.
Preparing to unpack .../23-ruby_1%3a3.0~exp1_amd64.deb ...
Unpacking ruby (1:3.0~exp1) ...
Selecting previously unselected package rake.
Preparing to unpack .../24-rake_13.0.6-2_all.deb ...
Unpacking rake (13.0.6-2) ...
Selecting previously unselected package ruby-net-telnet.
Preparing to unpack .../25-ruby-net-telnet_0.1.1-2_all.deb ...
Unpacking ruby-net-telnet (0.1.1-2) ...
Selecting previously unselected package ruby-webrick.
Preparing to unpack .../26-ruby-webrick_1.7.0-3ubuntu0.2_all.deb ...
Unpacking ruby-webrick (1.7.0-3ubuntu0.2) ...
Selecting previously unselected package ruby-xmlrpc.
Preparing to unpack .../27-ruby-xmlrpc_0.3.2-1ubuntu0.1_all.deb ...
Unpacking ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Selecting previously unselected package libruby3.0:amd64.
Preparing to unpack .../28-libruby3.0_3.0.2-7ubuntu2.12_amd64.deb ...
Unpacking libruby3.0:amd64 (3.0.2-7ubuntu2.12) ...
Selecting previously unselected package libsyntax2:amd64.
Preparing to unpack .../29-libsyntax2_2021.20210626.59705-1ubuntu0.3_amd64.deb

```

...
Unpacking libsynctex2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libteckit0:amd64.
Preparing to unpack .../30-libteckit0_2.5.11+ds1-1_amd64.deb ...
Unpacking libteckit0:amd64 (2.5.11+ds1-1) ...
Selecting previously unselected package libtexlua53:amd64.
Preparing to unpack .../31-libtexlua53_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libtexluajit2:amd64.
Preparing to unpack
.../32-libtexluajit2_2021.20210626.59705-1ubuntu0.3_amd64.deb ...
Unpacking libtexluajit2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libzzip-0-13:amd64.
Preparing to unpack .../33-libzzip-0-13_0.13.72+dfsg.1-1.1_amd64.deb ...
Unpacking libzzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Selecting previously unselected package xfonts-encodings.
Preparing to unpack .../34-xfonts-encodings_1%3a1.0.5-0ubuntu2_all.deb ...
Unpacking xfonts-encodings (1:1.0.5-0ubuntu2) ...
Selecting previously unselected package xfonts-utils.
Preparing to unpack .../35-xfonts-utils_1%3a7.7+6build2_amd64.deb ...
Unpacking xfonts-utils (1:7.7+6build2) ...
Selecting previously unselected package lmodern.
Preparing to unpack .../36-lmodern_2.004.5-6.1_all.deb ...
Unpacking lmodern (2.004.5-6.1) ...
Selecting previously unselected package preview-latex-style.
Preparing to unpack .../37-preview-latex-style_12.2-1ubuntu1_all.deb ...
Unpacking preview-latex-style (12.2-1ubuntu1) ...
Selecting previously unselected package t1utils.
Preparing to unpack .../38-t1utils_1.41-4build2_amd64.deb ...
Unpacking t1utils (1.41-4build2) ...
Selecting previously unselected package teckit.
Preparing to unpack .../39-teckit_2.5.11+ds1-1_amd64.deb ...
Unpacking teckit (2.5.11+ds1-1) ...
Selecting previously unselected package tex-gyre.
Preparing to unpack .../40-tex-gyre_20180621-3.1_all.deb ...
Unpacking tex-gyre (20180621-3.1) ...
Selecting previously unselected package texlive-binaries.
Preparing to unpack .../41-texlive-
binaries_2021.20210626.59705-1ubuntu0.3_amd64.deb ...
Unpacking texlive-binaries (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package texlive-base.
Preparing to unpack .../42-texlive-base_2021.20220204-1_all.deb ...
Unpacking texlive-base (2021.20220204-1) ...
Selecting previously unselected package texlive-fonts-recommended.
Preparing to unpack .../43-texlive-fonts-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-fonts-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-base.

```



```

Preparing to unpack .../44-texlive-latex-base_2021.20220204-1_all.deb ...
Unpacking texlive-latex-base (2021.20220204-1) ...
Selecting previously unselected package libfontbox-java.
Preparing to unpack .../45-libfontbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libfontbox-java (1:1.8.16-2) ...
Selecting previously unselected package libpdfbox-java.
Preparing to unpack .../46-libpdfbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libpdfbox-java (1:1.8.16-2) ...
Selecting previously unselected package texlive-latex-recommended.
Preparing to unpack .../47-texlive-latex-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-latex-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive-pictures.
Preparing to unpack .../48-texlive-pictures_2021.20220204-1_all.deb ...
Unpacking texlive-pictures (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-extra.
Preparing to unpack .../49-texlive-latex-extra_2021.20220204-1_all.deb ...
Unpacking texlive-latex-extra (2021.20220204-1) ...
Selecting previously unselected package texlive-plain-generic.
Preparing to unpack .../50-texlive-plain-generic_2021.20220204-1_all.deb ...
Unpacking texlive-plain-generic (2021.20220204-1) ...
Selecting previously unselected package tipa.
Preparing to unpack .../51-tipa_2%3a1.3-21_all.deb ...
Unpacking tipa (2:1.3-21) ...
Selecting previously unselected package texlive-xetex.
Preparing to unpack .../52-texlive-xetex_2021.20220204-1_all.deb ...
Unpacking texlive-xetex (2021.20220204-1) ...
Setting up fonts-lato (2.0-2.1) ...
Setting up fonts-noto-mono (20201225-1build1) ...
Setting up libwoff1:amd64 (1.0.2-1build4) ...
Setting up libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up libijs-0.35:amd64 (0.35-15build2) ...
Setting up libtexluaajit2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up libfontbox-java (1:1.8.16-2) ...
Setting up rubygems-integration (1.18) ...
Setting up libzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Setting up fonts-urw-base35 (20200910-1) ...
Setting up poppler-data (0.4.11-1) ...
Setting up tex-common (6.17) ...
update-language: texlive-base not installed and configured, doing nothing!
Setting up libjbig2dec0:amd64 (0.19-3build2) ...
Setting up libteckit0:amd64 (2.5.11+ds1-1) ...
Setting up libapache-pom-java (18-1) ...
Setting up ruby-net-telnet (0.1.1-2) ...
Setting up xfonts-encodings (1:1.0.5-0ubuntu2) ...
Setting up t1utils (1.41-4build2) ...
Setting up libidn12:amd64 (1.38-4ubuntu1) ...
Setting up fonts-texgyre (20180621-3.1) ...
Setting up libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.3) ...

```

```

Setting up ruby-webrick (1.7.0-3ubuntu0.2) ...
Setting up fonts-lmodern (2.004.5-6.1) ...
Setting up fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Setting up ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Setting up libsyntax2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up libgs9-common (9.55.0~dfsg1-0ubuntu5.13) ...
Setting up teckit (2.5.11+ds1-1) ...
Setting up libpdfbox-java (1:1.8.16-2) ...
Setting up libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) ...
Setting up preview-latex-style (12.2-1ubuntu1) ...
Setting up libcommons-parent-java (43-1) ...
Setting up dvisvgm (2.13.1-1) ...
Setting up libcommons-logging-java (1.2-2) ...
Setting up xfonts-utils (1:7.7+6build2) ...
Setting up libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up texlive-binaries (2021.20210626.59705-1ubuntu0.3) ...
update-alternatives: using /usr/bin/xdvi-xaw to provide /usr/bin/xdvi.bin
(xdvi.bin) in auto mode
update-alternatives: using /usr/bin/bibtex.original to provide /usr/bin/bibtex
(bibtex) in auto mode
Setting up lmodern (2.004.5-6.1) ...
Setting up texlive-base (2021.20220204-1) ...
/usr/bin/ucfr
/usr/bin/ucfr
/usr/bin/ucfr
/usr/bin/ucfr
mktexlsr: Updating /var/lib/texmf/ls-R-TEXLIVEDIST...
mktexlsr: Updating /var/lib/texmf/ls-R-TEXMFMAIN...
mktexlsr: Updating /var/lib/texmf/ls-R...
mktexlsr: Done.
tl-paper: setting paper size for dvips to a4:
/var/lib/texmf/dvips/config/config-paper.ps
tl-paper: setting paper size for dvipdfmx to a4:
/var/lib/texmf/dvipdfmx/dvipdfmx-paper.cfg
tl-paper: setting paper size for xdvi to a4: /var/lib/texmf/xdvi/XDvi-paper
tl-paper: setting paper size for pdftex to a4: /var/lib/texmf/tex/generic/tex-
ini-files/pdftexconfig.tex
Setting up tex-gyre (20180621-3.1) ...
Setting up texlive-plain-generic (2021.20220204-1) ...
Setting up texlive-latex-base (2021.20220204-1) ...
Setting up texlive-latex-recommended (2021.20220204-1) ...
Setting up texlive-pictures (2021.20220204-1) ...
Setting up texlive-fonts-recommended (2021.20220204-1) ...
Setting up tipa (2:1.3-21) ...
Setting up texlive-latex-extra (2021.20220204-1) ...
Setting up texlive-xetex (2021.20220204-1) ...
Setting up rake (13.0.6-2) ...
Setting up libruby3.0:amd64 (3.0.2-7ubuntu2.12) ...

```

```

Setting up ruby3.0 (3.0.2-7ubuntu2.12) ...
Setting up ruby (1:3.0~exp1) ...
Setting up ruby-rubygems (3.3.5-2ubuntu1.2) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for mailcap (3.70+nmu1ubuntu1) ...
Processing triggers for fontconfig (2.13.1-4.2ubuntu5) ...
Processing triggers for libc-bin (2.35-0ubuntu3.13) ...
/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_5.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm_debug.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc.so.2 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbb.so.12 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libhwloc.so.15 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_loader.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_0.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_opencl.so.0 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc_proxy.so.2 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero.so.0 is not a
symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero_v2.so.0 is not a
symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libumf.so.1 is not a symbolic link

Processing triggers for tex-common (6.17) ...
Running updmap-sys. This may take some time... done.
Running mktexlsr /var/lib/texmf ... done.
Building format(s) --all.
    This may take some time... done.

```